



DBDOME

Security Root-Cause Catalog

Domain: SEC (Security) | Vendor: PostgreSQL

This catalog lists the security root causes monitored by DBDOME for PostgreSQL. Entries are organized by Area -> Issue -> Root Cause. Each root cause has a unique identifier (Error/Root-Cause ID) used by the detection and alerting engine. Source view: rootcause.v_rootcauses.

15	Areas
67	Issues
598	Root Causes

Generated 03 Jul 2026



Access Control (ACC)

Active Transactions Present

Sessions are holding open (in-flight) transactions. Surfacing the active transactions - who is running them, against which database, how long they have been open, and the SQL text - is the starting point for spotting long-running / abandoned transactions, blocking, and anomalous or unauthorized activity.

SEC-SQL-ACC-011-RC02 Active transactions (excluding monitoring user)

All currently open / in-flight user transactions on the instance, excluding the dbdome monitoring login (dbd_mon_usr / dbdome_mon_usr). Surfaces who is holding a transaction, against which database, for how long, the transaction state and the SQL text - for spotting long-running or abandoned transactions, blocking, and anomalous activity.

Anomalous Transaction Activity

Running transactions show unusual patterns compared to expected baseline - unknown logins, unexpected programs, unusual databases, or suspicious query patterns that may indicate unauthorized access or compromised accounts.

SEC-SQL-ACC-010-RC07 After-hours transaction activity

Transactions are running outside normal business hours (before 06:00 or after 22:00 server local time). After-hours database activity is a common indicator of compromised credentials, insider threats, or unauthorized batch processes.

SEC-SQL-ACC-010-RC08 Privilege escalation attempts in transactions

Non-admin users are executing DDL or DCL statements (CREATE, ALTER, DROP, GRANT, REVOKE, DENY) that modify database structure or permissions. This may indicate privilege abuse, lateral movement, or a compromised application account attempting escalation.

SEC-SQL-ACC-010-RC09 Data exfiltration patterns in transactions

Active transactions show patterns consistent with data exfiltration: large result set extractions, bulk export operations (BCP, COPY, INTO OUTFILE), linked server/foreign data wrapper queries, or systematic table-by-table reads indicating data harvesting.

SEC-SQL-ACC-010-RC10 Same login active from multiple hosts

A single login name has concurrent active sessions originating from two or more different client hosts. This may indicate credential sharing, stolen credentials being used in parallel, or a brute-force attack that has succeeded.

SEC-SQL-ACC-010-RC11 Schema reconnaissance activity

Active transactions are querying system catalogs, metadata views, or information_schema tables to enumerate database objects, columns, permissions, or configurations. This is a common first step in database attacks — mapping the target before exploitation.

SEC-SQL-ACC-010-RC12 Dormant account suddenly active

An account that has had no login activity for an extended period (30+ days) is now executing transactions. Dormant accounts that suddenly become active are high-risk indicators of credential compromise or unauthorized reactivation.

SEC-SQL-ACC-010-RC13 Mass data modification in transactions

Active transactions are performing bulk UPDATE or DELETE operations affecting a large number of rows. This may indicate ransomware activity, data sabotage, unauthorized data manipulation, or a destructive script running against production.



Auditing (AUD)

Audit and Log Tampering

Detects attempts to cover tracks by issuing DML against audit tables, disabling audit settings, truncating log tables, modifying triggers, or clearing error buffers during an active session.

SEC-SQL-AUD-011-RC01 Direct DML executed on audit or log tables

INSERT, UPDATE, or DELETE statements target known audit or log tables (AUD\$, pgaudit logs, sys.fn_get_audit_file targets, mysql.general_log), indicating an attempt to modify or erase the activity record.

SEC-SQL-AUD-011-RC02 Audit settings disabled or downgraded mid-session

Commands such as ALTER SYSTEM SET log_statement=none, NOAUDIT, sp_configure audit level changes, or SET GLOBAL general_log=OFF are issued during an active session, reducing the audit coverage for subsequent actions.

SEC-SQL-AUD-011-RC03 Audit or log table truncated or dropped

A TRUNCATE or DROP statement targets an audit, log, or event-history table, destroying the evidentiary record of prior database activity.

SEC-SQL-AUD-011-RC04 Trigger disabled immediately before DML operation

An ALTER TABLE DISABLE TRIGGER or equivalent command is issued within the same session shortly before a DML statement on the same table, bypassing row-level audit or validation logic.

SEC-SQL-AUD-011-RC05 Error log or event buffer cleared during session

Commands such as sp_cycle_errorlog, ALTER SYSTEM SWITCH LOGFILE, FLUSH LOGS, or DBMS_SYSTEM.KSDWRT are executed, rotating or clearing the server error log to remove evidence of prior errors or suspicious activity.

Audit Logging Disabled

SEC-SQL-AUD-001-RC01 Global logging switch is disabled

The master configuration parameter controlling all logging (e.g., `logging\_collector`, `general\_log`) is set to "off" or "0" in the primary configuration file.

SEC-SQL-AUD-001-RC02 Missing audit plugin/extension

The required database audit plugin or extension is not installed or loaded into the server's shared memory.

SEC-SQL-AUD-001-RC03 Audit policy set to "NONE"

The audit policy is defined but explicitly configured to capture nothing (e.g., an empty rule set or a "disable" flag within the policy itself).

SEC-SQL-AUD-001-RC05 Service account permission failures

The database service account lacks write permissions to the designated log directory, causing the logging subsystem to fail silently or disable itself.

SEC-SQL-AUD-001-RC06 Performance-based disablement

Administrators intentionally disabled logging during a high-load event to reduce I/O or CPU overhead and failed to re-enable it.

SEC-SQL-AUD-001-RC07 Misconfigured log destination

Logging is enabled, but the destination is set to a non-existent path, `/dev/null`, or a remote syslog server that is unreachable, effectively dropping all logs.

SEC-SQL-AUD-001-RC08 Default "Secure by Default" settings

The database was installed with factory defaults, which often favor performance over observability, leaving auditing disabled until explicitly turned on.

SEC-SQL-AUD-001-RC09 Conflicting configuration files

A secondary configuration file (e.g., `conf.d/override.cnf`) is loading after the main config and overwriting the "enabled" setting with "disabled."

SEC-SQL-AUD-001-RC10 Plugin initialization failure

The audit plugin failed to initialize during startup (due to memory constraints or dependency errors), forcing the database to start without auditing.

**SEC-SQL-AUD-001-RC11 Container/Ephemeral storage misconfiguration**

In containerized environments, logs are written to an ephemeral path that is destroyed on restart, creating the appearance of disabled logging.

SEC-SQL-AUD-001-RC12 Audit process crash

The specific background process or thread responsible for writing audit buffers to disk has crashed or stalled, stopping log generation while the DB remains active.

Audit Trail Modification Risk**SEC-SQL-AUD-003-RC01 Insecure file permissions**

The operating system directory storing audit logs has `777` (world-writable) permissions, allowing any user to delete or modify log files.

SEC-SQL-AUD-003-RC02 Internal audit tables are mutable

Audit logs are stored in standard database tables that can be updated or truncated by users with `DBA` or `SUPERUSER` privileges.

SEC-SQL-AUD-003-RC03 Lack of File Integrity Monitoring (FIM)

No external security tool (like OSSEC or Tripwire) is monitoring the log files, so tampering goes undetected.

SEC-SQL-AUD-003-RC04 Service account ownership

The database service account owns the log files and the database process itself can be manipulated (e.g., via SQL injection or stored procedure) to overwrite them.

SEC-SQL-AUD-003-RC05 Missing "Append-Only" attributes

The filesystem does not use "append-only" attributes (`chattr +a` on Linux), allowing files to be truncated or deleted.

SEC-SQL-AUD-003-RC06 Audit logs not shipped remotely

Logs exist *only* on the database server. If an attacker gains root/admin access, they can wipe the only copy of the evidence.

SEC-SQL-AUD-003-RC08 Admin accounts shared/untracked

Multiple users share a generic "admin" login, making it impossible to attribute the deletion of logs to a specific individual.

SEC-SQL-AUD-003-RC09 Audit disablement capability

High-privileged users have the permission to turn off the auditing system temporarily, perform actions, and turn it back on without a trace of the disablement itself (if that event isn't logged elsewhere).

Database Reconnaissance Activity

Identifies schema enumeration behaviour typical of attackers mapping the database before exfiltration: excessive catalog queries, user/role listing by non-admin accounts, and mass column/permission discovery.

SEC-SQL-AUD-010-RC01 Excessive information_schema or catalog enumeration

A single session issues an unusually high number of queries against `information_schema`, `pg_catalog`, `sys.objects`, or `DBA_views` within a short window, consistent with automated schema mapping.

SEC-SQL-AUD-010-RC02 System principal and user enumeration by non-admin account

A non-administrative account queries `pg_roles`, `sys.server_principals`, `mysql.user`, `DBA_USERS`, or equivalent views listing all database accounts, indicating attacker reconnaissance of the privilege landscape.

SEC-SQL-AUD-010-RC03 Mass table and column listing within a single session

A session queries `information_schema.columns` or equivalent for all tables in a database in rapid succession, typically indicating an automated tool mapping the full column inventory for targeted exfiltration.

SEC-SQL-AUD-010-RC04 Stored procedure and function source code enumeration

A non-DBA account retrieves the source definitions of stored procedures, functions, or packages from `pg_proc`, `sys.sql_modules`, `information_schema.routines`, or `ALL_SOURCE`, suggesting reconnaissance of business logic.

SEC-SQL-AUD-010-RC05 Permission and privilege discovery queries

Queries against `HAS_TABLE_PRIVILEGE`, `fn_my_permissions`, `SESSION_PRIVS`, `information_schema.role_table_grants`, or equivalent reveal an account systematically mapping what it is allowed to access.



DDL Logging Disabled

SEC-SQL-AUD-005-RC01 DML-focused Audit Policy

The audit configuration focuses solely on data access ('SELECT', 'INSERT', 'UPDATE'), ignoring schema changes ('CREATE', 'ALTER', 'DROP').

SEC-SQL-AUD-005-RC02 Granularity settings

The logging level is set to "Statement" or "Query" but includes a filter that excludes specific keywords like 'ALTER' or 'DROP'.

SEC-SQL-AUD-005-RC03 Privileged User Exclusion

DDL is often performed by Superusers/DBAs, but the audit policy explicitly excludes the "admin" user to reduce log volume.

SEC-SQL-AUD-005-RC04 Logging "Write" vs "DDL"

The configuration enables "Write" auditing (data modification) but treats "DDL" (schema modification) as a separate category that was left disabled.

SEC-SQL-AUD-005-RC05 Deployment via Scripts

DDL changes are applied via automated deployment scripts using a specific service account that is whitelisted/excluded from auditing.

SEC-SQL-AUD-005-RC06 Temporary Table Exclusion

The system is configured to ignore DDL on temporary tables, but the filter is too broad and ignores real table changes.

SEC-SQL-AUD-005-RC07 Extension/Plugin Limitations

The installed auditing plugin only supports capturing DML events and lacks the hooks to intercept DDL commands.

SEC-SQL-AUD-005-RC08 Statement Truncation

DDL statements are logged, but the log buffer size is too small, truncating the query string so the actual 'ALTER' command is unreadable.

High Privilege User Suspicious Transactions

Suspicious database transactions executed by superusers, DBAs, or administrative roles that may indicate insider threat, credential compromise, or policy violation.

SEC-SQL-AUD-006-RC01 Privileged account executing DML outside maintenance window

A superuser or DBA account runs INSERT/UPDATE/DELETE during normal business hours or outside a scheduled window, signalling unplanned or unauthorized data modification.

SEC-SQL-AUD-006-RC02 Superuser running ad-hoc queries on sensitive tables

An admin account directly queries PII, credential, financial, or encryption-key tables outside an approved application context, indicating reconnaissance or exfiltration preparation.

SEC-SQL-AUD-006-RC03 High privilege account accessing schema outside designated scope

An administrative account that normally operates within a specific schema begins accessing schemas or databases outside its designated boundary, suggesting lateral movement.

SEC-SQL-AUD-006-RC04 Service account performing interactive or manual transactions

A service or application account (expected to run only parameterized queries) is observed in an interactive session running ad-hoc SQL, indicating possible credential theft.

SEC-SQL-AUD-006-RC05 Privilege escalation during active session

A session that started with standard privileges subsequently executes GRANT, SET ROLE, or other privilege-elevating commands within the same connection.

Inadequate Log Retention

SEC-SQL-AUD-002-RC01 Aggressive rotation configuration

Log rotation settings (time-based or size-based) are too aggressive, overwriting files faster than the compliance window (e.g., rotating every hour with only 10 files kept).

SEC-SQL-AUD-002-RC02 Insufficient disk space

The partition dedicated to logs is too small, forcing automated cleanup scripts or the database itself to delete older logs to prevent server downtime.



SEC-SQL-AUD-002-RC03 Lack of external archival

Logs are stored locally on the database server without a process (like `logrotate` or Splunk forwarders) to ship them to long-term storage (S3, cold storage) before deletion.

SEC-SQL-AUD-002-RC04 Default retention policies

The database is using default retention settings (often 7-30 days) which do not meet the 90+ day compliance requirement.

SEC-SQL-AUD-002-RC05 Memory-only logging

Auditing is configured to write to a memory buffer (ring buffer) which is lost upon database restart or when the buffer wraps around.

SEC-SQL-AUD-002-RC06 Misconfigured "Purge" jobs

Automated scheduled tasks (cron jobs or internal DB schedulers) are configured with the wrong retention calculation (e.g., `DELETE < GETDATE() - 7` instead of `90`).

SEC-SQL-AUD-002-RC07 File system quotas

The operating system user running the database has hit a file count or size quota, preventing new logs from being written or forcing immediate deletion of old ones.

SEC-SQL-AUD-002-RC08 Circular log overwrite

The auditing system is configured in a "circular" mode where it writes to a fixed-size file that overwrites itself once full, regardless of time elapsed.

SEC-SQL-AUD-002-RC09 Container restart data loss

In Kubernetes/Docker, logs are written to the container's stdout/stderr without a sidecar or logging driver to capture and persist them past the container's lifecycle.

SEC-SQL-AUD-002-RC10 Manual deletion by admins

Administrators manually clear log files to free up space during disk pressure incidents, creating gaps in the audit trail.

Long-Running Uncommitted Transactions

Flags transactions that remain open beyond a defined threshold without commit, idle-in-transaction sessions holding locks, and probe-and-revert rollback patterns consistent with ransomware or intentional lock abuse.

SEC-SQL-AUD-012-RC01 Transaction open beyond threshold with no activity

A session has an active transaction that has not been committed or rolled back for longer than a configurable threshold (e.g. 30 minutes) with no recent query activity, suggesting an abandoned, stuck, or deliberately held transaction.

SEC-SQL-AUD-012-RC02 Exclusive locks held on large tables for extended period

A session holds exclusive or access-exclusive locks on tables with a significant row count for an unusually long time, blocking other operations and potentially indicating intentional lock-based denial of service.

SEC-SQL-AUD-012-RC03 Idle-in-transaction session blocking downstream operations

A session in the idle in transaction state is holding locks that are blocking a queue of other waiting sessions, a pattern associated with application bugs, connection pool exhaustion, or deliberate resource starvation.

SEC-SQL-AUD-012-RC04 Large-volume rollback indicating probe-and-revert behaviour

A transaction that modified a large number of rows is rolled back, with the same pattern repeated multiple times within a session, consistent with an attacker probing data constraints or testing injection payloads without committing.

SEC-SQL-AUD-012-RC05 Savepoint abuse with repeated partial rollbacks

A session creates and rolls back to savepoints in rapid repeated cycles, which is abnormal application behaviour and may indicate automated probing of data state or attempts to exploit MVCC timing windows.

Missing Login Failure Logs

SEC-SQL-AUD-004-RC01 Logging level set to "Errors Only"

The logging verbosity is too low, capturing only system critical errors but ignoring authentication failures (which are often treated as "warnings" or "info").

SEC-SQL-AUD-004-RC02 "Successful Logins Only" filter

The audit policy is explicitly configured to log only *successful* connections to reduce noise, blinding the system to brute force attacks.

**SEC-SQL-AUD-004-RC03 Connection string errors**

Connection attempts that fail at the network/listener level (before reaching the database engine) are not logged by the DB, only by the firewall or OS.

SEC-SQL-AUD-004-RC04 Generic "Audit All" disabled

The global audit switch is on, but the specific subclass for "Session/Connection" events is disabled.

SEC-SQL-AUD-004-RC07 Internal vs External Authentication

The database relies on external auth (LDAP/Kerberos). Failures happen at the Identity Provider level and are never sent to the database to be logged.

SEC-SQL-AUD-004-RC08 Application-side suppression

Middleware or connection poolers catch login exceptions and retry silently without the database ever logging a distinct "failure" event for each retry.

Risky Transaction Execution

High-impact operations such as bulk data exports, mass deletions, production DDL changes, or access to credential and encryption tables that represent significant data-loss or integrity risk.

SEC-SQL-AUD-008-RC01 Bulk data export or mass SELECT on sensitive tables

A single session issues SELECT queries returning unusually large row counts from tables classified as sensitive, consistent with data exfiltration.

SEC-SQL-AUD-008-RC02 Mass DELETE or TRUNCATE without WHERE clause

A DELETE with no predicate or a TRUNCATE statement executes against a non-empty table, risking irreversible data loss.

SEC-SQL-AUD-008-RC03 DDL statements executed in production by non-DBA accounts

CREATE, ALTER, or DROP statements are executed by accounts that do not hold an approved DBA role, violating change-management controls.

SEC-SQL-AUD-008-RC04 Transactions accessing credential or encryption key tables

Any account other than the designated key-management service queries tables that store hashed passwords, API keys, or encryption key material.

SEC-SQL-AUD-008-RC05 Cross-schema transactions outside application scope

A session joins or accesses tables from two or more schemas that the associated application has no business reason to combine, suggesting unauthorized data correlation.

Schema and DML Activity Tracking

Tracks every CREATE/ALTER/DROP and every INSERT/UPDATE/DELETE/TRUNCATE recorded by the SQL Server Audit subsystem. The detection auto-provisions the server audit and per-database audit specifications when missing so coverage is consistent across all online databases including master.

SEC-SQL-AUD-015-RC01 Recent schema changes or DML detected via SQL Server Audit

Returns every schema-change (CREATE/ALTER/DROP) and DML (INSERT/UPDATE/DELETE/TRUNCATE) event recorded by the target database's audit subsystem over the last 24 hours. Auto-provisions audit infrastructure on first run for each vendor.

Security Configuration Change During Active Session

Detects runtime changes to security-relevant server or session parameters: ALTER SYSTEM, sp_configure, trigger disabling, audit parameter changes, and enabling dangerous features such as xp_cmdshell or UTL_FILE.

SEC-SQL-AUD-013-RC01 Security-relevant ALTER SYSTEM or SET executed by non-admin

A non-superuser or non-DBA account issues ALTER SYSTEM, SET SESSION, or sp_configure commands that change authentication, SSL, logging, or access-control parameters, undermining the server security posture.

SEC-SQL-AUD-013-RC02 Dangerous feature enabled during session

Commands enabling xp_cmdshell, UTL_FILE directory access, DBMS_SCHEDULER OS jobs, or LOAD DATA INFILE are issued during an active session, opening OS-level execution vectors from within the database.

SEC-SQL-AUD-013-RC03 Trigger disabled or dropped on monitored table

ALTER TABLE DISABLE TRIGGER, DROP TRIGGER, or equivalent commands target tables that carry audit, validation, or row-history triggers, removing controls that enforce data integrity or change tracking.

**SEC-SQL-AUD-013-RC04 Audit or logging parameters changed mid-session**

Session-level commands reduce or disable query logging, error reporting, or audit verbosity (e.g. SET log_min_duration_statement=-1, NOAUDIT ALL, SET GLOBAL slow_query_log=OFF), creating a blind spot for subsequent activity.

SEC-SQL-AUD-013-RC05 Network or authentication configuration modified at runtime

Commands that alter pg_hba.conf, firewall rules, linked-server definitions, or remote authentication providers are issued at runtime, potentially opening new access paths that bypass existing controls.

SQL Injection Runtime Indicators

Detection of live SQL injection patterns in active sessions: tautologies, stacked queries, UNION probing, time-delay functions, and comment-based obfuscation embedded in query text.

SEC-SQL-AUD-009-RC01 Tautology patterns detected in active query text

Active queries contain always-true conditions such as OR 1=1, OR 'a'='a', or OR 1>0 that are characteristic of classic SQL injection payloads used to bypass WHERE-clause filtering.

SEC-SQL-AUD-009-RC02 Stacked or chained query execution detected

A single query string contains multiple statement terminators followed by additional SQL commands, indicating a stacked-query injection where the attacker appends arbitrary statements to a legitimate request.

SEC-SQL-AUD-009-RC03 UNION-based probing across unknown tables

Queries contain UNION SELECT constructs referencing system catalog tables or column counts inconsistent with the legitimate application, indicating UNION-based injection used to extract schema or data.

SEC-SQL-AUD-009-RC04 Time-delay injection functions detected in active queries

Queries contain time-delay calls such as pg_sleep, WAITFOR DELAY, SLEEP(), or DBMS_LOCK.SLEEP, which are the hallmark of blind time-based SQL injection used to infer data without visible output.

SEC-SQL-AUD-009-RC05 Comment-based obfuscation or encoding in query text

Queries contain inline SQL comments (/**/, --), hex-encoded strings, CHAR() concatenation, or URL-encoded characters used to obfuscate injection payloads and bypass input validation or WAF rules.

Successful database authentication monitoring

Capture and verify successful database authentication (logon) events.

SEC-SQL-AUD-021-RC01 Successful database authentication events captured

Confirms DBDOME captures successful database logons (authentication events) across all supported engines: reads the live current-session view (V\$SESSION / dm_exec_sessions / pg_stat_activity / PROCESSLIST / syssessions) and, on Oracle & SQL Server, also the audit trail of successful logons. Verify by simply logging in to the database.

Unknown Transaction Patterns

Transactions that do not match known application baselines: unrecognized query signatures, unexpected source connections, or off-hours activity suggesting unauthorized access.

SEC-SQL-AUD-007-RC01 Transactions from unregistered application signatures

SQL traffic originates from an application_name, client program, or connection tag not present in the approved application registry, suggesting a rogue or shadow tool.

SEC-SQL-AUD-007-RC02 Query patterns not matching known application baseline

Queries exhibit structural signatures (table combinations, column access, JOIN depth) that do not appear in the established workload fingerprint, indicating manually crafted or injected SQL.

SEC-SQL-AUD-007-RC03 Connections from unexpected network addresses

Active database sessions originate from IP ranges or hostnames not listed in the approved connection whitelist, indicating an unauthorized client.

SEC-SQL-AUD-007-RC04 Off-hours transaction activity from non-automated accounts

Human-owned accounts generate database activity during nights, weekends, or holidays when no scheduled batch jobs are expected, consistent with unauthorized access.

SEC-SQL-AUD-007-RC05 Execution of previously unseen stored procedures or functions

A stored procedure or function that has never appeared in historical workload logs is called, potentially indicating a backdoor or



newly injected code object.

User and Role Manipulation During Session

Identifies CREATE USER, role-grant, password-change, and external database-link creation commands issued mid-session, which are typical of privilege persistence, backdoor account creation, or lateral movement setup.

SEC-SQL-AUD-014-RC01 New user or login created during application session

A CREATE USER, CREATE LOGIN, or CREATE ROLE statement is executed during what appears to be a normal application session, indicating possible backdoor account creation for persistent re-entry.

SEC-SQL-AUD-014-RC02 Account added to privileged role mid-session

A GRANT ROLE, ALTER SERVER ROLE ADD MEMBER, or GRANT DBA command elevates an existing account to a highly privileged role (sysadmin, DBA, superuser) outside of a formal change-management process.

SEC-SQL-AUD-014-RC03 Password changed on another account during active session

An ALTER USER ... PASSWORD or equivalent command changes the credential of an account other than the currently logged-in user, potentially resetting a dormant account for re-activation.

SEC-SQL-AUD-014-RC04 Database link or synonym to external system created

A CREATE DATABASE LINK, CREATE SYNONYM, sp_addlinkedserver, or CREATE FOREIGN DATA WRAPPER command establishes a connection pathway to an external database, which could be used to exfiltrate data or pivot to another system.

SEC-SQL-AUD-014-RC05 Account unlocked or expiry removed on dormant account

An ALTER USER ... ACCOUNT UNLOCK, ENABLE LOGIN, or password-expiry-removal command reactivates a previously locked or expired account, which may be an attacker restoring access to a previously compromised credential.



Authentication (AU)

Default Admin Accounts Enabled

SEC-SQL-AU-001-RC01 Installation defaults not changed

Database installation scripts create superuser/admin accounts with known names by default. Administrators forget or skip the post-installation hardening step to rename, disable, or change credentials.

SEC-SQL-AU-001-RC02 Installation mode misalignment

Choosing a restricted authentication mode during installation disables the default superuser account, but later switching to a more permissive mode without explicitly setting a strong password leaves the account empty or unchanged.

SEC-SQL-AU-001-RC03 Database not in protected network

Assumption that the database is internal-only. Default accounts are left active because administrators believe external access is restricted, creating a false sense of security.

SEC-SQL-AU-001-RC04 Lack of post-deployment hardening checklist

Organizations deploy databases following application-focused checklists that omit security hardening steps, particularly default account remediation.

SEC-SQL-AU-001-RC05 Knowledge gap on account purposes

DBAs don't understand which default accounts are system-critical versus removable sample or demo accounts, leading to inconsistent remediation of default credentials.

SEC-SQL-AU-001-RC06 Incomplete migration procedures

When moving from development to production, default accounts created in dev environments are not reviewed or removed during promotion.

SEC-SQL-AU-001-RC07 Recovery procedure reliance

Some DBAs intentionally leave default accounts enabled to have a recovery path if regular admin accounts are locked out or credentials are lost.

SEC-SQL-AU-001-RC08 Multi-vendor environment complexity

Organizations running multiple database platforms lack a unified hardening framework, causing inconsistency in which defaults are addressed.

SEC-SQL-AU-001-RC09 No integration with identity provisioning

Default accounts are not tracked in identity access management systems, making them invisible to access reviews and governance processes.

SEC-SQL-AU-001-RC10 Account privilege escalation over time

Default admin accounts are initially left "for testing" but become production accounts as applications depend on them, and nobody revisits the decision.

SEC-SQL-AU-001-RC11 Inadequate change control

Post-installation account changes lack change order tracking, so when environments are rebuilt or cloned, original default credentials return without verification.

SEC-SQL-AU-001-RC12 Assumption of "unused" protection

DBAs believe unused accounts don't pose risk if network access is restricted, overlooking lateral movement threats within the network.

Dormant/Unused Accounts

SEC-SQL-AU-006-RC01 No activity monitoring

Databases lack or don't use login activity logging; dormant accounts are invisible without manual audits.

SEC-SQL-AU-006-RC02 Offboarding process incomplete

When employees depart, access removal is assigned but not verified to completion; accounts remain active indefinitely.

SEC-SQL-AU-006-RC03 Service account lifecycle unknown

Service accounts created for applications/integrations are never tracked; when the application is decommissioned, the account persists.



SEC-SQL-AU-006-RC04 Contractor account cleanup forgotten

Temporary contractor or vendor accounts created with expiration dates, but expiration is not enforced; accounts become dormant.

SEC-SQL-AU-006-RC05 Temporary project access never revoked

Database accounts created for specific projects are marked as temporary but never removed after project completion.

SEC-SQL-AU-006-RC06 Test/dev accounts migrated to production

Development and testing accounts created in non-prod environments are cloned to production and left untouched.

SEC-SQL-AU-006-RC07 No formal account inventory

Organizations don't maintain authoritative lists of who should have database access, making dormant accounts invisible.

SEC-SQL-AU-006-RC08 HR/identity integration absent

No integration between HR/identity systems and database access management; terminated employees' database accounts are not automatically disabled.

SEC-SQL-AU-006-RC09 Quarterly reviews too infrequent

Access reviews happen once per year or less; accounts can be dormant for months before detection.

SEC-SQL-AU-006-RC10 Shared service accounts misclassified

Accounts used by applications are treated as user accounts; when applications are decommissioned, accounts are forgotten.

SEC-SQL-AU-006-RC11 Account ownership unclear

Accounts lack documented owners; when owners leave or departments reorganize, ownership trail is lost and accounts become orphaned.

SEC-SQL-AU-006-RC12 No automated dormancy detection

Database tools that identify unused accounts aren't deployed or their reports aren't acted upon.

SEC-SQL-AU-006-RC13 Privilege accumulation over time

Dormant accounts often retain elevated privileges from their original roles; privilege isn't reduced just because the account is unused.

SEC-SQL-AU-006-RC14 Audit trails insufficient

Login auditing doesn't capture enough detail to distinguish "never used since creation" from "used once years ago."

SEC-SQL-AU-006-RC15 Remediation effort underestimated

Removing dormant accounts is deferred due to fear of breaking "hidden" dependencies, so accounts remain enabled indefinitely.

Empty/Blank Passwords Allowed

SEC-SQL-AU-002-RC01 Initialization flag usage

Database initialization flag or mode creates the default superuser account with an empty password intentionally (for local testing convenience), but the insecure configuration remains in place when deployed to production.

SEC-SQL-AU-002-RC03 Installation interruption

Installation processes that require password entry for admin accounts may be skipped or terminated early, leaving accounts with null passwords.

SEC-SQL-AU-002-RC04 Test account forgotten in production

Temporary test accounts created with blank passwords during development are not removed before promotion to production.

SEC-SQL-AU-002-RC05 Password never set in configuration

Default superuser account does not require a password for OS-authenticated local connections, but when remote access is later enabled, no password has been set, leaving the account exposed.

SEC-SQL-AU-002-RC06 Identity provisioning tool failure

IAM systems that create database accounts may fail on the final password-setting step while leaving the account created and enabled with empty password set flag.

SEC-SQL-AU-002-RC07 Configuration file permission bypass

Administrators create accounts with empty passwords intentionally to store credentials in secured configuration files, believing the file permissions compensate for null passwords.



SEC-SQL-AU-002-RC09 Application provisioning defaults

Application deployment frameworks automatically create database accounts with empty passwords if password parameters are omitted in templates.

SEC-SQL-AU-002-RC10 LDAP/directory integration incomplete

Organizations planning to use LDAP authentication create database accounts with null passwords until LDAP integration is completed, but integration is never finished.

SEC-SQL-AU-002-RC11 Batch user creation scripts

DBA scripts for bulk account creation use empty passwords as placeholders expecting them to be changed later, but cleanup never happens.

SEC-SQL-AU-002-RC12 No pre-deployment security scanning

Accounts with empty/null passwords are not detected by deployment validation tools before the environment goes live.

Remote OS Authentication Enabled

SEC-SQL-AU-004-RC01 PostgreSQL ident authentication default

PostgreSQL's default `pg_hba.conf` for remote connections (IPv4/IPv6) sometimes specifies `ident` authentication, delegating trust to the ident server or OS identification.

SEC-SQL-AU-004-RC03 OS service account access

Applications run as OS service accounts (e.g., the database service account), and the database trusts that OS identity for authentication, bypassing password checks.

SEC-SQL-AU-004-RC04 OS-integrated authentication trust configuration

Database configured to trust OS-level or directory-service authentication for remote access, assuming that OS or domain security is equivalent to database security.

SEC-SQL-AU-004-RC05 OS authentication module delegation

Database configured to delegate authentication to an OS-level module (PAM, SSPI, or similar), including for remote connections where OS identity cannot be reliably verified.

SEC-SQL-AU-004-RC06 Kerberos/SSPI authentication without encryption

Windows SSPI or Kerberos authentication enabled remotely but without requiring channel encryption, assuming token freshness equals authentication security.

SEC-SQL-AU-004-RC07 Cross-system domain trusts

Multiple database servers trust a single directory server (LDAP, AD) for authentication, but access controls aren't enforced per-database.

SEC-SQL-AU-004-RC08 No validation of OS identity

Systems accepting OS usernames from clients over the network without cryptographic verification of actual OS user identity.

SEC-SQL-AU-004-RC09 Legacy Unix ident server

Database configured to use the legacy ident protocol (port 113) for network authentication, which provides no cryptographic verification of client identity.

SEC-SQL-AU-004-RC10 Assume-inside-firewall security model

Configuration assumes all network clients are trusted because they're "inside the firewall," enabling OS auth without additional controls.

SEC-SQL-AU-004-RC11 LDAP with weak connection security

LDAP authentication for databases configured over unencrypted connections, allowing credential interception and replay.

SEC-SQL-AU-004-RC12 No OS authentication logging

Database logs don't capture details about OS authentication source, making audits of remote OS auth impossible.

Trust Authentication Enabled

SEC-SQL-AU-007-RC01 Default trust-based local authentication

Database installation creates default authentication rules that allow unauthenticated local or loopback connections, and these permissive defaults are left unchanged in production.



SEC-SQL-AU-007-RC02 Administrator convenience during setup

DBAs enable `trust` authentication temporarily for initial setup and testing (to avoid password requirements), then forget to disable it.

SEC-SQL-AU-007-RC03 Peer authentication misunderstood as secure

DBAs confuse `peer` (secure for local Unix sockets) with `trust` (unconditional) and leave `trust` enabled for TCP/IP thinking it's equivalent.

SEC-SQL-AU-007-RC04 Development environment settings

Database in development or testing uses permissive authentication (no password required) for ease of access; settings are replicated to production via configuration management without security review.

SEC-SQL-AU-007-RC05 Remote access addition overlooked

When remote access is enabled on the database, the existing permissive local authentication rules are not replaced with password-based or certificate-based rules, allowing any user on localhost to connect without credentials.

SEC-SQL-AU-007-RC06 Incomplete hardening documentation

Organizations implementing database security guidance focus on external access controls but overlook local authentication; permissive local authentication rules are never hardened.

SEC-SQL-AU-007-RC07 Configuration file management tools

Ansible, Terraform, or other infrastructure-as-code templates use default database authentication configuration without customization; automated deployments inherit insecure defaults.

SEC-SQL-AU-007-RC08 No security scanning of pg_hba.conf

Deployment validation tools don't check for `trust` entries or alert administrators to the risk.

SEC-SQL-AU-007-RC09 Recovery procedure reliance

DBAs intentionally keep `trust` for loopback to recover from lost superuser passwords, not recognizing container/VM host access risks.

SEC-SQL-AU-007-RC10 Container/Kubernetes secrets practice

In containerized database deployments, passwordless local authentication is enabled because the environment is assumed to be isolated, ignoring that other containers or pod sidecars on the same host can access the database.

SEC-SQL-AU-007-RC11 Single-node assumption

Developers assume the database is single-machine; passwordless local authentication seems safe until replication, clustering, or remote backup tools are added and expose the permissive configuration.

SEC-SQL-AU-007-RC12 Administrator password forgotten or locked

When the superuser password is lost and the account is locked, `trust` mode becomes an unintended recovery path that's never cleaned up afterward.

SEC-SQL-AU-007-RC13 Documentation templates outdated

Organization-internal database deployment guides copy old documentation that includes permissive authentication rules without explaining the risk or suggesting removal.

SEC-SQL-AU-007-RC14 No configuration drift detection

Database authentication configurations are not regularly scanned for unauthorized or overly permissive authentication entries.

Weak Password Policy

SEC-SQL-AU-003-RC01 Default configuration never modified

Databases ship with minimal or no password complexity requirements, and administrators never configure stronger policies after installation.

SEC-SQL-AU-003-RC02 Complexity enforcement disabled intentionally

Password complexity enforcement policy disabled intentionally to ease administration or reduce user friction, without recognizing the security trade-off of allowing weak passwords.

SEC-SQL-AU-003-RC03 Legacy system requirements

Old applications cannot handle special characters or uppercase/lowercase requirements, so weak policies are set to accommodate legacy apps.



SEC-SQL-AU-003-RC04 Regulatory compliance misinterpretation

NIST 800-63 recommendations (longer passphrases without complexity) are misinterpreted as no complexity required at all.

SEC-SQL-AU-003-RC05 Complexity vs. length tradeoff avoided

Organizations reject "complex" password policies thinking they'll switch to passphrase-based systems, but never implement the alternative.

SEC-SQL-AU-003-RC06 Performance concerns

Administrators believe password hashing/validation overhead is significant and weaken policies to reduce CPU cost, without evidence of actual impact.

SEC-SQL-AU-003-RC07 User friction avoidance

DBAs and admins reduce complexity to minimize support calls and password reset requests, prioritizing convenience over security.

SEC-SQL-AU-003-RC08 Development environment settings bleeding to production

Dev databases with relaxed policies are cloned or templated to production without policy review.

SEC-SQL-AU-003-RC09 No centralized policy definition

In multi-database environments, each database has different policies or none at all, with no central enforcement mechanism.

SEC-SQL-AU-003-RC10 Password manager unavailability

Absence of organizational password management tools leads administrators to assume users cannot handle complex passwords and weaken requirements.

SEC-SQL-AU-003-RC11 Lack of monitoring tools

Organizations without compromised credential monitoring don't perceive the risk of weak passwords and continue relaxed policies.

SEC-SQL-AU-003-RC12 Policy documentation missing

No written policy exists, so individual DBAs enforce their own (often minimal) standards.

SEC-SQL-AU-003-RC13 Account creation automation without validation

Scripts that create accounts batch-mode use weak password templates without policy enforcement.



Authentication (AUTH)

Default Credentials

SEC-VEC-AUTH-002-RC06

Peer/ident authentication without password fallback

PostgreSQL configured with peer/ident authentication for OS-level users, bypassing password requirements.



Authorization (AUTHZ)

Database Access & Privilege Inventory

Inventory of database access: server logins, database users, role memberships and explicit permission grants across all user databases, for access review and least-privilege auditing.

SEC-SQL-AUTHZ-001-RC01

Database access & privilege inventory (logins, roles, grants)

Enumerates, for every online user database on the instance, each database principal mapped to its server login, the database roles it belongs to, and its explicit permission grants - a Server -> Database -> Login -> roles & grants inventory for access review.



Authorization (AZ)

Cross-Database Ownership Chaining

SEC-SQL-AZ-005-RC01 Insufficient understanding of ownership chaining mechanics

Database architects and administrators do not fully understand how ownership chaining bypasses permission checks across databases; they enable the feature without recognizing the security implications.

SEC-SQL-AZ-005-RC02 Convenience over security principle

Cross-database ownership chaining simplifies permission management by allowing stored procedures to access objects in other databases without explicit permission grants; this operational convenience outweighs security concerns.

SEC-SQL-AZ-005-RC03 Legacy application design assuming ownership chaining

Older applications were architected with the assumption that ownership chaining would be enabled; disabling it would require application refactoring, so the setting remains enabled.

SEC-SQL-AZ-005-RC04 Unclear ownership across multiple databases

When database ownership is complex (different owners across different databases, or ownership changes over time), administrators enable cross-database chaining to simplify the permission model rather than clarifying and rationalizing ownership.

SEC-SQL-AZ-005-RC05 Multi-database applications without explicit permission architecture

Applications that legitimately access multiple databases across a server do not implement explicit cross-database permissions; instead, ownership chaining is relied upon.

SEC-SQL-AZ-005-RC06 Migration from single-database to multi-database environments

Organizations originally designed for a single database later expand to multiple databases; rather than redesigning permission structures, cross-database chaining is enabled to preserve existing privilege logic.

SEC-SQL-AZ-005-RC08 Lack of audit or monitoring of cross-database access

No logging or alerting occurs when a stored procedure in one database accesses objects in another; unintended privilege escalations are not detected.

SEC-SQL-AZ-005-RC09 Testing configurations carried forward to production

Test or development environments enable cross-database chaining for convenience; these settings are replicated to production without review.

SEC-SQL-AZ-005-RC10 Third-party vendor requirements or recommendations

ERP, CRM, or other enterprise applications require or recommend enabling cross-database chaining; organizations enable it at the server level to support multiple vendor applications.

SEC-SQL-AZ-005-RC11 Incomplete understanding of risk when owner differs

Administrators may not realize that cross-database chaining only forms if both databases are owned by the same login; when this condition is met (often unintentionally), privilege escalation becomes possible.

SEC-SQL-AZ-005-RC12 Database isolation principle not enforced

Organizations lack a architectural principle that databases should be isolated from each other; without this principle, cross-database access is seen as acceptable.

Excessive Administrative Privileges

SEC-SQL-AZ-001-RC01 Lack of Role-Based Access Control (RBAC) implementation

Organizations without mature RBAC systems grant permissions on an ad hoc basis rather than aligning them to defined job roles, resulting in superuser privileges being distributed widely across the user population.

SEC-SQL-AZ-001-RC02 Principle of least privilege not enforced

No governance mechanism ensures users have only the minimum permissions needed for their tasks, allowing administrative privileges to accumulate beyond necessity.

SEC-SQL-AZ-001-RC03 Developer environments influence production practices

Developers working in development/test environments with elevated permissions (such as db_owner or DBA) write code assuming those privileges, then request the same elevated access in production to avoid refactoring.



SEC-SQL-AZ-001-RC04 Time pressure and perceived operational convenience

DBAs and administrators grant DBA or superuser roles to satisfy urgent requests faster than identifying and granting specific required privileges, treating elevated access as a shortcut to enable functionality.

SEC-SQL-AZ-001-RC05 Inadequate permission analysis during application deployment

Developers and application teams fail to precisely determine which privileges are needed during deployment scripting, instead requesting broad administrative rights to ensure "everything works."

SEC-SQL-AZ-001-RC06 Multi-schema and cross-database application requirements

Applications that legitimately access multiple schemas or databases across the database server are granted elevated privileges rather than being re-architected with explicit per-object permissions.

SEC-SQL-AZ-001-RC07 Third-party application vendor requirements

ISVs and third-party software vendors explicitly require or recommend DBA-level privileges for their applications, and organizations grant these without evaluating the actual necessity.

SEC-SQL-AZ-001-RC08 Absence of periodic access reviews and certification

No formal process exists to regularly audit who has administrative privileges, why they have them, and whether the justification still applies.

SEC-SQL-AZ-001-RC09 Employee role transitions without privilege adjustment

When employees change roles or responsibilities, their old administrative privileges are not revoked, creating accumulation over time.

SEC-SQL-AZ-001-RC10 Legacy applications with hardcoded privilege assumptions

Older applications were built assuming superuser-level database access; refactoring these to use constrained permissions is expensive, so elevated access persists.

SEC-SQL-AZ-001-RC11 Knowledge gaps and lack of training

DBAs, application teams, and management lack education about the principle of least privilege, security risks of excessive permissions, and techniques for granular access control.

SEC-SQL-AZ-001-RC12 Vendor default configurations

Database installations from major vendors ship with default accounts and elevated privilege structures; these are not hardened during initial setup.

SEC-SQL-AZ-001-RC13 No centralized identity and access governance

Database access is managed independently from enterprise identity systems (Active Directory, LDAP), creating silos where cross-system privilege consistency cannot be enforced.

SEC-SQL-AZ-001-RC14 Shared service and application accounts

Multiple applications or users share single accounts with elevated privileges, making it impossible to attribute actions and enforce individual least privilege.

File System Access Privilege

SEC-SQL-AZ-004-RC01 Application requirements for file export functionality

Applications legitimately need to export data to files; filesystem access privileges are granted to application service accounts to support this feature.

SEC-SQL-AZ-004-RC02 Lack of secure alternatives or awareness of alternatives

Administrators are unaware of safer alternatives such as restricted export directories, parameterized export functions, or file-based I/O limitations; they grant full FILE privilege instead.

SEC-SQL-AZ-004-RC03 Business intelligence and reporting tool requirements

BI tools and reporting applications request FILE access to write query results to files; rather than implementing tools with built-in safe export mechanisms, FILE privilege is granted.

SEC-SQL-AZ-004-RC04 Inadequate file operation path restrictions

Database file operation path restrictions are not properly configured to constrain file import/export to safe directories, granting uncontrolled filesystem access to accounts with file operation privileges.



SEC-SQL-AZ-004-RC05 Default configuration not hardened during installation

Database installations use default settings that do not restrict file operations; these defaults are not changed during initial setup.

SEC-SQL-AZ-004-RC06 Developer convenience during development

File privileges are granted to development accounts for testing; these elevated permissions are then copied to production accounts without being removed.

SEC-SQL-AZ-004-RC07 Troubleshooting and ad-hoc data extraction

DBAs grant FILE privilege for temporary troubleshooting (dumping data to files for analysis) or ad-hoc data extraction requests; the privilege is not revoked after the task completes.

SEC-SQL-AZ-004-RC08 Third-party tool or vendor requirements

ETL tools, backup solutions, or vendor applications explicitly require FILE or COPY privileges; organizations grant these without evaluating whether safer alternatives exist.

SEC-SQL-AZ-004-RC09 Historical privilege accumulation

File privileges were granted years ago for purposes that no longer apply; they are never audited or revoked.

SEC-SQL-AZ-004-RC10 Insufficient monitoring of FILE privilege usage

No auditing or monitoring occurs to track which users exercise FILE privilege, for what purposes, or with what frequency; privilege grants cannot be justified or revoked.

SEC-SQL-AZ-004-RC11 Shared service accounts with FILE privilege

Batch job and application service accounts share FILE privilege; removing the privilege would require refactoring multiple systems.

SEC-SQL-AZ-004-RC12 Configuration management errors

Database provisioning scripts or configuration templates incorrectly grant FILE privilege to broader user sets than intended.

SEC-SQL-AZ-004-RC13 Privilege grants during emergency scenarios

During incident response or data recovery situations, FILE privileges are temporarily granted; they are not revoked after the emergency passes.

SEC-SQL-AZ-004-RC14 Misunderstanding of attack vectors

Administrators do not fully appreciate that FILE privilege allows attackers to overwrite system files, load malicious libraries, or exfiltrate sensitive OS files (e.g., /etc/passwd).

Grant Option Abuse

SEC-SQL-AZ-006-RC01 Overly broad delegation of grant rights

Administrators grant users the WITH GRANT OPTION to allow them to delegate permissions, without clear restrictions on what permissions can be delegated or to whom.

SEC-SQL-AZ-006-RC02 Inadequate understanding of WITH GRANT OPTION implications

Administrators and users do not fully understand that WITH GRANT OPTION allows privilege escalation; they view it simply as "letting users grant other users access" without recognizing the security risks.

SEC-SQL-AZ-006-RC03 Convenience for distributed access management

Rather than centralizing permission administration, administrators grant WITH GRANT OPTION to department leads or team managers to distribute permissions; this delegates security control without oversight.

SEC-SQL-AZ-006-RC04 No approval workflow for permission delegation

Users with WITH GRANT OPTION can grant privileges to other users without any review, audit, or approval process; unvetted privilege escalations occur freely.

SEC-SQL-AZ-006-RC05 Inadequate monitoring of grant activity

No logging or alerting occurs when users exercise WITH GRANT OPTION to grant permissions to others; privilege escalation chains are never detected.

SEC-SQL-AZ-006-RC06 Lack of restrictions on re-delegation

Users with WITH GRANT OPTION can grant permissions to others, who can then grant those permissions further; uncontrolled privilege cascading occurs.

SEC-SQL-AZ-006-RC07 Grant option on sensitive privileges

WITH GRANT OPTION is granted on high-risk privileges (e.g., SUPERUSER, ALTER SYSTEM, CREATE ROLE) that should never



be delegated.

SEC-SQL-AZ-006-RC08 Default role configurations including grant option

Predefined roles created during database setup include WITH GRANT OPTION without documented justification; administrators inherit these settings without review.

SEC-SQL-AZ-006-RC09 Privilege escalation through role membership

Users granted WITH GRANT OPTION on a role can grant that role to themselves or others, effectively escalating their own privileges.

SEC-SQL-AZ-006-RC10 Separation of duties violations

A single user with both elevated privileges AND WITH GRANT OPTION can grant those same privileges to other users, creating uncontrolled privilege proliferation.

SEC-SQL-AZ-006-RC11 Implicit grant options in template scripts

Database provisioning scripts or configuration templates include WITH GRANT OPTION in GRANT statements; these are copied across multiple databases without review.

SEC-SQL-AZ-006-RC12 Compliance framework gaps

Organizations lack policies or controls aligned with frameworks (SOX, HIPAA, ISO 27001) that restrict the ability to delegate privileges.

SEC-SQL-AZ-006-RC13 Inherited permissions from legacy systems

Users have WITH GRANT OPTION from configurations created years ago; no one questions whether the original justification still applies.

SEC-SQL-AZ-006-RC14 Confusion between intent and capability

Administrators may not realize that granting a privilege WITH GRANT OPTION to a low-privilege user is a security risk; they assume the low-privilege user will not abuse the capability.

Orphaned User Accounts

SEC-SQL-AZ-003-RC01 Absence of user lifecycle management automation

Organizations lack formal processes or tools to automatically deprovision accounts when employees leave, contractors end engagements, or systems are decommissioned.

SEC-SQL-AZ-003-RC02 Manual offboarding processes with gaps

Exit procedures exist but are incomplete, error-prone, or not followed consistently; database account deprovisioning is often overlooked while network and application access are revoked.

SEC-SQL-AZ-003-RC03 Delayed or forgotten deprovisioning

Deprovisioning requests are submitted but not executed in a timely manner, particularly during periods of high staff turnover or when the DBA handling the request leaves without completing it.

SEC-SQL-AZ-003-RC04 Lack of integration between HR systems and database access

Database access is not synchronized with the authoritative source of truth for employee status (HR system, LDAP, Active Directory); when an employee is terminated in HR, the database account is not automatically disabled.

SEC-SQL-AZ-003-RC05 No reconciliation process between database accounts and authorized users

No regular audit compares accounts in the database against the current roster of employees and contractors; dormant and orphaned accounts are never identified.

SEC-SQL-AZ-003-RC06 Service and application accounts without ownership tracking

Accounts created for applications, batch jobs, or services lack clear ownership documentation; when an application is decommissioned, no one knows which database accounts to remove.

SEC-SQL-AZ-003-RC07 Contractor and vendor accounts without explicit end dates

Third-party accounts are created without documented expiration dates; they persist indefinitely even after the contractor relationship ends.

SEC-SQL-AZ-003-RC08 Shared database accounts and credentials

Multiple users share a single database account (often through shared passwords), making it impossible to identify which accounts can be safely removed without disrupting someone's work.



SEC-SQL-AZ-003-RC09 Cross-database and application account complexity

Users have accounts across multiple databases and systems; when they leave, accounts in some systems are deprovisioned while others are forgotten.

SEC-SQL-AZ-003-RC10 Legacy account creation without documentation

Accounts created many years ago lack ownership documentation or audit trails; administrators cannot determine why they exist or who should use them.

SEC-SQL-AZ-003-RC11 Database environment fragmentation

Organizations maintain multiple database environments (production, staging, development) with separate account management; accounts in lower environments are never cleaned up.

SEC-SQL-AZ-003-RC12 Lack of privileged access management (PAM) systems

Without centralized privileged account management, individual DBAs manage credentials and account lifecycles inconsistently.

SEC-SQL-AZ-003-RC13 Application default accounts never updated

Applications ship with default service accounts that are provisioned but never changed; when applications are decommissioned, these accounts remain.

SEC-SQL-AZ-003-RC14 Failed or incomplete migration processes

During database migrations or major system changes, legacy accounts are migrated along with data; documentation of why they are needed is lost.

SEC-SQL-AZ-003-RC15 Assumption that low-privilege accounts are harmless

Orphaned read-only accounts are left active because they are perceived as low-risk; this creates an expanded attack surface and complicates audit trails.

rootcause Role Excessive Access

SEC-SQL-AZ-002-RC01 Misunderstanding of rootcause role scope

Administrators do not fully understand that permissions granted to rootcause automatically apply to every user on the database, including future users and accounts they do not intend to provision.

SEC-SQL-AZ-002-RC02 Quick-fix troubleshooting approach

When specific users report permission errors, administrators grant the required permission to rootcause rather than identifying and fixing the actual required privilege for the specific user or role.

SEC-SQL-AZ-002-RC03 Temporary workarounds becoming permanent

Initial troubleshooting grants to rootcause are intended as temporary solutions but remain in place permanently once the immediate issue is resolved.

SEC-SQL-AZ-002-RC04 Legacy database practices

Older database installations had rootcause granted broad permissions to simplify management; these permissions were never audited or removed during subsequent upgrades.

SEC-SQL-AZ-002-RC05 Insufficient permission auditing on rootcause

No regular reviews occur to audit what permissions rootcause currently holds and whether those permissions align with organizational security policies.

SEC-SQL-AZ-002-RC06 Default view permissions in system objects

Many databases grant rootcause SELECT on system views and metadata tables by default; administrators extend this to user tables without recognizing the risk.

SEC-SQL-AZ-002-RC07 Confusion between schema and database-level rootcause

Administrators may grant specific intended permissions at one level but not fully understand the inheritance and scope across the entire database or server.

SEC-SQL-AZ-002-RC08 Testing/development configuration copied to production

Development or test databases grant rootcause broad access for convenience; these configurations are replicated to production without review.

SEC-SQL-AZ-002-RC09 Bulk permission scripts without proper restriction

Configuration management scripts or deployment automation grant permissions at scale without explicit filtering to exclude



rootcause.

SEC-SQL-AZ-002-RC10 Lack of role-based alternative

No well-defined roles for common user categories exist; instead of creating "read-only_users" or "application_readers" roles, administrators grant rootcause permissions.

SEC-SQL-AZ-002-RC11 Incompletely implemented permission revocation

Attempts to restrict rootcause permissions fail or are incomplete; revocation statements have syntax errors, incomplete scope, or are not applied to all objects.

SEC-SQL-AZ-002-RC12 Tool or vendor recommendations that are unsafe

Third-party tools or vendors recommend granting specific permissions to rootcause as the simplest configuration approach.

SEC-SQL-AZ-002-RC13 Emergency access during incidents

During production outages or incidents, rootcause permissions are temporarily granted to enable rapid troubleshooting; these are never revoked once normalcy is restored.



Configuration (CFG)

Database Links Unsecured

SEC-SQL-CFG-006-RC01 Static Credential Storage

Database links configured with fixed usernames/passwords stored unencrypted in link definition; attacker with read access to data dictionary views can extract credentials

SEC-SQL-CFG-006-RC02 Overprivileged Link Credentials

Link configured to connect as highly privileged user (DBA, admin role) to remote database; anyone accessing the local database can implicitly access remote database as admin

SEC-SQL-CFG-006-RC03 rootcause Database Link Exposure

Link created as rootcause rather than private; all database users inherit ability to use link rather than restricted set

SEC-SQL-CFG-006-RC04 Privilege Escalation via Link

Low-privilege user on local database (read-only access) discovers rootcause link to remote database with high privileges; escalates access cross-database

SEC-SQL-CFG-006-RC05 Inadequate Access Control

No validation in link definition that local users with access to link should have equivalent privileges on remote database

SEC-SQL-CFG-006-RC06 Cross-Database Privilege Inheritance

Assumption that local database privilege restrictions apply to remote database; actually grants unrestricted access to remote tables within remote user's privileges

SEC-SQL-CFG-006-RC07 Unvalidated Dynamic Queries

Link used in dynamic SQL or application code without parameterization; allows SQL injection through database link to execute arbitrary remote commands

SEC-SQL-CFG-006-RC08 Remote Database Exploitation

Compromised local database account uses link to compromise remote database; removes network boundary to breach remote systems

SEC-SQL-CFG-006-RC09 Credential Exposure via Monitoring

Link credentials visible in database alert logs, monitoring tools, or query plans; exposed through overly permissive system catalog access

SEC-SQL-CFG-006-RC10 Long-Lived Credentials

Link credentials never rotated; if exposed in any context (decommissioned server, backup leak), attacker has permanent access

SEC-SQL-CFG-006-RC11 Scope Confusion

Organization unaware that link credential applies across all schemas/users; assumes capability is restricted to specific applications

SEC-SQL-CFG-006-RC12 Network Segmentation Bypass

Application-layer network controls don't apply to database links; link bypasses firewall rules between database tiers

SEC-SQL-CFG-006-RC13 Linked Server Configuration Persistence

Database links configured during migration or initial setup, persists even after application requirements change

SEC-SQL-CFG-006-RC14 Missing Encryption

Link credentials transmitted unencrypted over network; exposed to any network sniffer between databases (especially in cloud/shared networks)

SEC-SQL-CFG-006-RC15 Mutual Authentication Gaps

Links don't validate remote database identity; susceptible to man-in-the-middle attacks where attacker masquerades as remote database

SEC-SQL-CFG-006-RC16 Catalog-Level Exposure

Data dictionary views exposing link connection strings to users who need only the functional capability (not configuration details)



Debug/Trace Flags Enabled

SEC-SQL-CFG-005-RC02 Vendor/Support Recommendation

Vendor support or consulting recommends specific trace flag for compatibility or workaround; enabled without documented plan to remove

SEC-SQL-CFG-005-RC03 Undocumented Historical Change

Trace flag enabled during system migration or upgrade; change documentation lost or DBA who made change departed organization

SEC-SQL-CFG-005-RC04 Perceived Performance Optimization

DBA reads that a trace flag or parameter improves temporary tablespace or checkpoint behavior; enables it without baseline testing or comparative metrics

SEC-SQL-CFG-005-RC05 Dangerous Flag Combinations

Multiple trace flags enabled over time; combination of flags creates unexpected behaviors or conflicts not documented

SEC-SQL-CFG-005-RC06 Experimentation in Production

DBA testing different trace flag values directly in production to find optimal setting; tests never cleaned up

SEC-SQL-CFG-005-RC07 One-Off Debugging Enabled Persistently

Trace flag enabled to diagnose specific deadlock or query issue; becomes permanent when logs stored for historical reference

SEC-SQL-CFG-005-RC08 Configuration Drift

Trace flags configured in one environment (dev/test) and propagated to production via infrastructure-as-code without security review

SEC-SQL-CFG-005-RC09 Undocumented Server Requirements

Third-party application silently depends on particular trace flag being enabled; application breaks if flag removed despite original need unclear

SEC-SQL-CFG-005-RC10 Verbose Logging for "Security"

Trace flags enabling query/transaction logging enabled as "security measure" but left on 24/7 causing performance and disk space issues

SEC-SQL-CFG-005-RC12 Lazy Optimization

DBAs use trace flags as shortcut instead of properly tuning queries, indexes, or parameters; technical debt accumulates

SEC-SQL-CFG-005-RC13 Lack of Audit Trail

No mechanism to track when/why trace flags were enabled; difficult to justify continued use or identify candidates for removal

SEC-SQL-CFG-005-RC14 Misunderstanding Hidden Features

DBA confuses trace flags with standard configuration parameters; treats trace flags as permanent settings rather than debugging tools

In-Database Code Execution Enabled

SEC-SQL-CFG-004-RC02 Performance Optimization

Legacy code paths use CLR for complex algorithmic operations rather than T-SQL; disabling CLR would require extensive refactoring

SEC-SQL-CFG-004-RC03 Third-Party Tool Dependencies

SSISDB (Integration Services), Reporting Services, or replication features require CLR to function; enabling CLR is collateral consequence

SEC-SQL-CFG-004-RC04 Unsigned Assembly Bypass

CLR strict security disabled to avoid certificate signing requirements on assemblies; enabled feature persists even if original assemblies removed

SEC-SQL-CFG-004-RC05 Lack of Assembly Vetting

CLR assemblies deployed without security review; code review process doesn't identify untrusted or overprivileged operations

SEC-SQL-CFG-004-RC06 Privilege Escalation via CLR

Attacker with limited database privileges creates malicious CLR assembly to execute with database engine's process privileges



(often highly elevated)

SEC-SQL-CFG-004-RC07 Sandbox Escape

CLR "safe" assemblies contain vulnerabilities allowing escape from .NET sandbox; enables file system access or process execution despite SAFE designation

SEC-SQL-CFG-004-RC08 Registry Modification

CLR code can modify Windows registry; attacker can alter system configuration, disable antivirus, or create persistence mechanisms

SEC-SQL-CFG-004-RC09 Network-Based Attacks

CLR assemblies can initiate arbitrary network connections (if written poorly) to exfiltrate data or download additional payloads

SEC-SQL-CFG-004-RC10 External Library Access

CLR code can load additional .NET libraries that provide unrestricted OS access, bypassing intended security boundaries

SEC-SQL-CFG-004-RC11 Post-Exploitation Persistence

Once CLR enabled, attacker can create malicious procedure to persist access across restarts; difficult to detect without assembly auditing

SEC-SQL-CFG-004-RC12 Insufficient Configuration Segregation

Trustworthy flag or CLR strict security configured at instance level; affects all databases despite only one or two needing the feature

SEC-SQL-CFG-004-RC13 Monitoring Gaps

No alerting on CLR assembly creation or execution; malicious CLR activity remains undetected

Local File Loading Enabled

SEC-SQL-CFG-002-RC02 Legitimate Data Loading Requirements

Applications genuinely require LOAD DATA LOCAL INFILE for bulk data imports (ETL processes, initial data seeding); feature left enabled after deployment

SEC-SQL-CFG-002-RC03 Inadequate Server-Side Controls

Focus on enabling the feature during development for testing but no implementation of compensating controls (file restrictions, rate limiting) before production use

SEC-SQL-CFG-002-RC04 Unclear Default Behavior

Documentation confusion about whether local_infile is enabled by default; DBAs/developers assume it's secure by default when it's not

SEC-SQL-CFG-002-RC06 Unvalidated User Input

If application accepts filename or path from users/queries without validation, attackers can exploit local_infile to read arbitrary system files

SEC-SQL-CFG-002-RC08 Information Disclosure

Enabled local_infile allows reading database configuration files, application source code, or other server files that contain secrets or architectural information

SEC-SQL-CFG-002-RC09 Implicit Trust Model

Configuration assumes all clients connecting to the database are trusted; local_infile enabled system-wide rather than per-session or per-user

SEC-SQL-CFG-002-RC10 Client-Side Exploitation

Man-in-the-middle or compromised client application can trigger unsolicited LOAD DATA LOCAL requests to read server-side files from client perspective

SEC-SQL-CFG-002-RC11 Backup Retention

local_infile enabled during initial bulk loading and never disabled; remains active through multiple backup/restore cycles and environment promotions

SEC-SQL-CFG-002-RC12 Third-Party Application Defaults

Vendor-supplied applications (analytics tools, middleware) set local_infile=ON in their deployment templates without organization-specific review



SEC-SQL-CFG-002-RC13 Insufficient Documentation

DBA instructions or runbooks don't explicitly mention disabling local_infile; it's treated as a minor configuration detail

OS Command Execution Enabled

SEC-SQL-CFG-001-RC03 Administrative Troubleshooting Convenience

DBAs enable OS command execution for ad-hoc system administration tasks (file operations, command execution) rather than using proper automation tools or secure alternatives.

SEC-SQL-CFG-001-RC04 Inadequate Change Control

OS command execution is enabled during development/testing and never disabled when promoting to production due to missing security review in change control process.

SEC-SQL-CFG-001-RC05 SQL Injection Vulnerability

Applications with SQL injection vulnerabilities allow attackers to enable OS command execution if it is available; attacker gains operating system access through the database.

SEC-SQL-CFG-001-RC07 Lack of Monitoring and Auditing

Absence of query logging or auditing means attackers can enable and use OS command execution without detection or alerting.

SEC-SQL-CFG-001-RC08 Knowledge Gap on Alternatives

DBAs unaware of secure alternatives (external process invocation, ETL tools, external procedures) default to OS command execution for system integration tasks.

SEC-SQL-CFG-001-RC09 Privilege Escalation via Weak Credentials

Compromised low-privilege database login can use OS command execution to pivot to operating system administrator access if the database service account is overprivileged.

SEC-SQL-CFG-001-RC10 Lateral Movement Capability

Once attacker gains OS command execution access, they can use operating system commands to execute lateral movement tools for network reconnaissance and further compromise.

SEC-SQL-CFG-001-RC11 Persistence Mechanisms

OS command execution allows creation of scheduled tasks, registry/cron modifications, or service installation for maintaining post-exploitation persistence.

SEC-SQL-CFG-001-RC12 Backup Restoration from Compromised Environments

Production databases restored from backups created in compromised lower-level environments may retain OS command execution enabled configuration.

Sample/Test Databases Present

SEC-SQL-CFG-003-RC01 Default Installation Artifacts

Vendor installation scripts create sample databases (pubs, northwind, test) by default; scripts are run without review of what's being created

SEC-SQL-CFG-003-RC02 Development Environment Cloning

Production database environments copied from development/test environments; sample schemas from dev environment remain present

SEC-SQL-CFG-003-RC03 Backup Restoration Error

Backups taken from systems with sample databases are restored to production without pre-restore pruning of unnecessary objects

SEC-SQL-CFG-003-RC04 Migration Process Gaps

Database migration tools (like Microsoft Data Migration Assistant) transfer all objects including test schemas; migrators don't filter schemas by importance

SEC-SQL-CFG-003-RC05 Knowledge Loss During Transitions

Original DBA/team that provisioned environment leaves organization; new team unaware of which databases are legitimate vs. test artifacts

SEC-SQL-CFG-003-RC06 Inadequate Cleanup Procedures

Post-deployment scripts don't include cleanup steps to remove sample databases after server is hardened



SEC-SQL-CFG-003-RC07 Application Configuration Pointing to Wrong Database

Applications misconfigured to point at test/sample database instead of production database; remaining sample database "in case" needed

SEC-SQL-CFG-003-RC08 Demonstration and Training Use

Sample databases left in place for new DBA training or vendor demonstrations; forgotten after training is complete

SEC-SQL-CFG-003-RC09 Documentation Failures

Server documentation and asset inventory don't flag which databases are production vs. non-production; no single source of truth on what should exist

SEC-SQL-CFG-003-RC10 Insufficient RBAC Controls

Sample databases left with default permissions; no restriction preventing users from connecting to or querying test databases

SEC-SQL-CFG-003-RC11 Compliance Scope Confusion

Assumption that test/sample databases are exempt from data protection policies; sample data treated casually despite containing realistic PII from examples

SEC-SQL-CFG-003-RC12 Automated Provisioning Templates

Infrastructure-as-code or automated deployment scripts include sample database creation steps that persist through production deployments

SEC-SQL-CFG-003-RC13 Weak Object Naming Conventions

Sample databases not named distinctly enough to be easily identified (e.g., "test" is ambiguous); gets confused with legitimate test systems

SEC-SQL-CFG-003-RC14 Multi-Tenant Instance Management

On shared database instances, sample databases from one customer/application remain even as others are removed



Data Modeling (DAT)

Unencrypted At-Rest

SEC-VEC-DAT-002-RC08 PostgreSQL pgvector without host-level or TDE encryption

pgvector embeddings inherit PostgreSQL row-level storage; without pgcrypto extension or host-level encryption, vectors are stored in plaintext in PostgreSQL's data files.



Encryption (ENC)

Data-in-Transit Unencrypted

SEC-SQL-ENC-001-RC01 Default Configuration Not Changed

Many database engines have TLS/SSL encryption disabled by default in their initial configuration files. Organizations deploy databases without reviewing and updating these defaults.

SEC-SQL-ENC-001-RC02 Missing Encryption Parameter in Connection Strings

Applications and client tools use connection strings without the vendor-specific encryption parameter, resulting in unencrypted connections by default.

SEC-SQL-ENC-001-RC03 Backward Compatibility Requirements

Legacy applications were built against database clients that lack TLS 1.2+ support. Enabling encryption would break these applications, forcing a choice between security and functionality.

SEC-SQL-ENC-001-RC04 Client Driver Version Mismatch

Older versions of database drivers (ODBC, JDBC, client libraries) do not support encryption or only support deprecated protocols. Applications cannot upgrade drivers without extensive testing and potential code changes.

SEC-SQL-ENC-001-RC05 Cost and Performance Concerns

Organizations avoid encryption due to perceived CPU overhead (typically 5-10% per source) and the cost of hardware upgrades or HSM integration, without proper ROI analysis.

SEC-SQL-ENC-001-RC06 Lack of DBA Encryption Knowledge

Database administrators lack the expertise to properly configure TLS/SSL certificates, manage certificate lifecycles, or troubleshoot encryption-related issues. They avoid implementing encryption to prevent breaking production systems.

SEC-SQL-ENC-001-RC07 Certificate Management Complexity

Organizations have not established processes for generating, storing, distributing, and renewing SSL/TLS certificates across all database clients and servers. This operational complexity deters implementation.

SEC-SQL-ENC-001-RC08 Self-Signed Certificate Issues

Certificate validation failures occur when using self-signed certificates without proper trust chains. Organizations disable certificate verification (`'trustServerCertificate=true'`) rather than resolving root CA trust problems.

SEC-SQL-ENC-001-RC09 Network Architecture Assumptions

Teams assume database communication occurs only on trusted internal networks (VLANs, firewalls) and therefore believe encryption is unnecessary, missing VLAN hopping, rogue access point, and internal threat vectors.

SEC-SQL-ENC-001-RC10 Lack of Security Requirements

No security policy or compliance mandate exists requiring encrypted connections. Encryption is treated as optional rather than mandatory.

SEC-SQL-ENC-001-RC11 Application Middleware Bypass

Applications connect directly to the database, bypassing middleware and service meshes that could enforce encryption, because they were not designed with this architecture.

SEC-SQL-ENC-001-RC12 Testing and Development Environment Carryover

Unencrypted connections configured for testing environments are carried over to production without modification.

SEC-SQL-ENC-001-RC13 Inadequate Monitoring and Visibility

Organizations lack tools to detect or audit whether connections are encrypted, making unencrypted connections invisible until a security audit reveals them.

SEC-SQL-ENC-001-RC14 Competing Priorities in Development Cycles

Encryption configuration is deprioritized in favor of feature delivery and bug fixes during development sprints.



Obsolete TLS Versions

SEC-SQL-ENC-002-RC01 Legacy Application Dependency on TLS 1.0/1.1

Older applications and client libraries explicitly require TLS 1.0 or 1.1 due to age (pre-2010 development). Disabling these protocols breaks application functionality.

SEC-SQL-ENC-002-RC02 Deprecated Client Libraries in Use

Applications use SQL Native Client, OLEDB (SQLOLEDB), or other deprecated database drivers that only support TLS 1.0/1.1 and cannot be upgraded due to application vendor end-of-life.

SEC-SQL-ENC-002-RC03 .NET Framework Constraints

Applications use .NET Framework versions prior to 4.6 which do not natively support TLS 1.2. Applications cannot be upgraded due to Windows OS version limitations or vendor support requirements.

SEC-SQL-ENC-002-RC04 Java/JDK Version Limitations

Older Java applications run on JDK versions that do not support TLS 1.2+ without additional configuration or patches that the organization has not applied.

SEC-SQL-ENC-002-RC06 Operating System Default Restrictions

The OS has disabled TLS 1.0/1.1 at the Schannel level (Windows registry), but the database server was installed before this change, and reinstallation is not feasible.

SEC-SQL-ENC-002-RC07 Load Balancer and Infrastructure TLS Settings

Network load balancers, firewalls, or proxies between clients and databases are configured to allow TLS 1.0/1.1 for "compatibility," and the database inherits these weaker settings.

SEC-SQL-ENC-002-RC08 Lack of TLS 1.2 Configuration Knowledge

DBAs do not know how to properly configure TLS 1.2 as the minimum version without breaking existing applications and choose to leave weaker versions enabled as a workaround.

SEC-SQL-ENC-002-RC09 Configuration Oversight

TLS 1.0 and 1.1 were disabled in registry/config by security hardening scripts but the database server was not restarted, or configuration was done on some nodes in a cluster but not all.

SEC-SQL-ENC-002-RC10 Intentional Compatibility Decisions

Organizations made an explicit decision to support legacy clients for business reasons (customer contracts, managed service agreements) without implementing compensating controls.

SEC-SQL-ENC-002-RC11 No Automated Configuration Audit

There is no recurring audit or monitoring to detect when TLS 1.0/1.1 becomes enabled via patches, configuration drifts, or application updates.

SEC-SQL-ENC-002-RC12 Database Version Limitations

Older database versions do not support disabling TLS 1.0/1.1 at the database level and require OS-level configuration.

SEC-SQL-ENC-002-RC13 Hybrid Cloud Connectivity

Organizations connecting on-premises databases to cloud resources via VPN or dedicated connections may leave TLS 1.0/1.1 enabled due to VPN appliance or cloud service requirements.

SEC-SQL-ENC-002-RC14 Vendor Testing Incompleteness

The organization tested upgrading to TLS 1.2-only with a small subset of applications but did not complete organization-wide testing, leaving legacy protocols enabled pending full migration.

Transparent Data Encryption Disabled

SEC-SQL-ENC-003-RC01 Not Aware of TDE Feature

Database administrators are unaware that TDE exists in their database version or believe TDE is only available in premium/enterprise editions and assume they cannot use it.

SEC-SQL-ENC-003-RC02 Licensing Restrictions

TDE requires premium edition licensing or add-on options. Organizations use standard editions or lack budget for the required license upgrades.



SEC-SQL-ENC-003-RC04 Key Management System Not Available

Organizations lack an external key management system (HSM, AWS KMS, Azure Key Vault, HashiCorp Vault) or knowledge of how to set one up. TDE requires a secure key store.

SEC-SQL-ENC-003-RC05 Performance Impact Concerns

Organizations measured or heard about TDE performance overhead (8-12% for TDE, typically 5-6% per encrypted column) and concluded the cost was not justified.

SEC-SQL-ENC-003-RC06 Database Design Incompatibility

Existing databases have design issues (unsupported data types, indexing constraints, compatibility with encrypted columns) that prevent TDE implementation without major refactoring.

SEC-SQL-ENC-003-RC07 Application Testing Not Completed

TDE was evaluated but applications were not thoroughly tested with encrypted databases. Failures in testing (incompatible queries, timeouts) caused implementation to be abandoned.

SEC-SQL-ENC-003-RC08 Backup and Recovery Complexity

Organizations do not understand that backups encrypted with TDE require the encryption key to restore, adding operational complexity for disaster recovery and off-site backup storage.

SEC-SQL-ENC-003-RC09 Administrative Overhead

DBAs perceive TDE as requiring extensive configuration, monitoring, and key rotation processes that they are not equipped to manage.

SEC-SQL-ENC-003-RC11 High Availability and Replication Issues

TDE adds complexity to failover, replication, and cluster configurations. Organizations avoid TDE to simplify high-availability operations.

SEC-SQL-ENC-003-RC12 Cloud Database Settings

In cloud environments (RDS, Azure SQL), TDE may be disabled by default, and organizations do not enable it because they assume the cloud provider's encryption is sufficient (confusion with AWS/Azure managed encryption at the storage level).

SEC-SQL-ENC-003-RC13 Development/Test Database Assumption

Organizations assume TDE is only needed in production and do not enable it in development databases, then never backfill to production due to risk aversion.

SEC-SQL-ENC-003-RC14 Lack of Compliance Driver

The organization operates in a vertical without strict data protection regulations (e.g., not HIPAA, PCI-DSS, GDPR-governed), so there is no compliance mandate to encrypt data at rest.

SEC-SQL-ENC-003-RC15 Previous Failed Implementation Attempt

An earlier attempt to enable TDE caused unexpected downtime or performance degradation, and the organization has not attempted it since.

SEC-SQL-ENC-003-RC16 Wallets and Key Storage Issues

TDE encryption keys or key storage files are kept on the same server as the database, defeating the security purpose. Organizations disable TDE because proper external key management is too complex to implement.

Unencrypted Backups

SEC-SQL-ENC-004-RC01 Feature Not Enabled at Backup Time

Backup procedures do not include the encryption option. Each vendor requires vendor-specific syntax or tools for backup encryption, and organizations use default backup commands without enabling it.

SEC-SQL-ENC-004-RC02 Backup Software Doesn't Support Encryption

Legacy backup software (Veritas, CommVault, or in-house scripts) does not support database backup encryption or was configured before encryption features were added.

SEC-SQL-ENC-004-RC03 Key Management for Backups Not Established

Even if encryption is configured, the organization has not established secure key management for backup decryption. Backup encryption keys are stored alongside backup files or not backed up, making recovery impossible.



SEC-SQL-ENC-004-RC05 Performance Impact on Backup Windows

Encrypting backups adds CPU overhead and I/O load, extending backup windows beyond acceptable limits. The organization chose speed over security.

SEC-SQL-ENC-004-RC06 Backup Storage Assumptions

Backups are stored in what the organization considers secure locations (isolated networks, locked rooms) and the team assumes backups do not need encryption because physical access is restricted.

SEC-SQL-ENC-004-RC07 Regulatory Requirement Not Mapped to Backups

Compliance policies (GDPR, HIPAA, PCI-DSS) require data encryption but the organization interprets this as applying only to the live database, not backup files.

SEC-SQL-ENC-004-RC08 Off-Site Backup Storage Not Recognized as High Risk

Backups sent to cloud storage or third-party sites are not encrypted, even though this increases exposure. The organization assumes the third party's security is sufficient.

SEC-SQL-ENC-004-RC09 Backup Recovery Testing Not Performed

Encryption was never tested end-to-end because the organization does not regularly test backup recovery. When tested, it failed due to missing keys or incompatible encryption formats.

SEC-SQL-ENC-004-RC10 Certificate and Key Expiration on Backups

Backup encryption certificates or keys expire, and the organization did not renew them before expiration. Backups taken after certificate expiration may be unrestoreable.

SEC-SQL-ENC-004-RC11 Database Encryption Complexity Misunderstood

Organizations with Transparent Data Encryption (TDE) enabled mistakenly believe backups are automatically encrypted because TDE encrypts the database. They do not realize that backup encryption requires separate configuration.

SEC-SQL-ENC-004-RC12 No Centralized Key Management Service

The organization lacks a centralized key management system (KMS), making it difficult to issue, store, and manage encryption keys for backups. Encryption is not implemented due to this operational gap.

SEC-SQL-ENC-004-RC13 Multiple Backup Destinations Not Coordinated

Backups are taken to local storage, network shares, and cloud storage with different tools and configurations. Some destinations support encryption, others do not, and the organization defaults to no encryption for consistency.

SEC-SQL-ENC-004-RC14 Backup Retention Policy Overrides Encryption

Backup retention policies store backups for years (regulatory requirement), and encrypting older backups is seen as too complex. The organization keeps backups unencrypted to simplify management.

SEC-SQL-ENC-004-RC15 Developer and QA Databases Using Production Backups

Unencrypted backup files are regularly restored to development and QA environments for testing, and the organization fears encryption will complicate this workflow.

SEC-SQL-ENC-004-RC16 Backup Scripts Written Before Encryption Features Existed

In-house backup scripts were written years ago and have not been updated despite new encryption capabilities in the database platform.

Weak Encryption Keys/Ciphers

SEC-SQL-ENC-005-RC01 Default/Legacy Cipher Suites Not Disabled

Database configurations enable weak cipher suites (RC4, DES, 3DES, MD5-signed certificates, CBC mode) for backward compatibility. Organizations have not explicitly disabled these weaker algorithms.

SEC-SQL-ENC-005-RC02 Short Key Lengths Selected for Performance

Organizations configured encryption algorithms with short key lengths (128-bit instead of 256-bit AES, 1024-bit RSA instead of 2048-bit) to minimize CPU overhead.

SEC-SQL-ENC-005-RC03 Deprecated Algorithm Still in Use

The organization is using older encryption algorithms (Blowfish, DES, SHA-1) that were appropriate when configured but have since been deprecated due to known vulnerabilities, and the configuration has never been updated.



SEC-SQL-ENC-005-RC04 Certificate Signed with Weak Hash

SSL/TLS certificates are signed with MD5 or SHA-1 instead of SHA-256, representing a legacy certificate that has never been replaced.

SEC-SQL-ENC-005-RC05 Self-Signed Certificates with Weak Parameters

Self-signed certificates were generated with weak parameters (1024-bit RSA, SHA-1 signature) using command-line defaults instead of explicit strong key specifications.

SEC-SQL-ENC-005-RC06 Key Generation with Insufficient Entropy

Encryption keys or certificates were generated in a way that resulted in weak key material (e.g., Debian weak key generation vulnerability, keys generated from predictable seeds).

SEC-SQL-ENC-005-RC07 Cipher Suite Configuration Not Updated

Database cipher suite configuration inherits weak defaults from OS-level TLS/SSL libraries. Even though stronger ciphers exist, the organization has not explicitly configured the database to prefer them.

SEC-SQL-ENC-005-RC08 No Cipher Suite Hardening Standard

There is no organizational standard for which cipher suites should be enabled. Each database is configured differently, and older configurations include weak suites.

SEC-SQL-ENC-005-RC09 RSA Key Exchange Still Enabled

Configurations allow RSA-based key exchange, which is vulnerable to decryption if the private key is compromised. TLS 1.2+ should prefer ECDHE for forward secrecy, but RSA is still enabled.

SEC-SQL-ENC-005-RC10 CBC Mode Still Allowed in Configuration

Cipher suites using CBC (Cipher Block Chaining) mode are allowed, which are vulnerable to specific attacks (BEAST, padding oracle). AEAD modes (GCM) are preferred but not enforced.

SEC-SQL-ENC-005-RC11 TLS 1.2 Allows Weak Ciphers

TLS 1.2 itself does not mandate strong ciphers; servers can negotiate weak ones. Organizations running TLS 1.2 but allowing weak cipher suites have false confidence in their security.

SEC-SQL-ENC-005-RC12 Key Rotation Policy Not Implemented

Encryption keys have never been rotated, and weak keys from the initial setup remain in use years later. There is no process to retire old keys and generate new ones.

SEC-SQL-ENC-005-RC13 Asymmetric Key Weak Parameters

RSA keys used for encryption or digital signatures use key sizes below 2048 bits, or Elliptic Curve keys use weak curves.

SEC-SQL-ENC-005-RC14 No Monitoring of Negotiated Ciphers

The organization does not monitor or audit which cipher suites are actually being negotiated in real TLS connections. Weak ciphers might be in use even if stronger ones are configured.

SEC-SQL-ENC-005-RC15 Compliance Requirements Misunderstood

PCI-DSS, HIPAA, and other standards specify "strong encryption" but do not define exact algorithms or key lengths. Organizations interpret this loosely and use weaker algorithms they believe still meet "strong" criteria.

SEC-SQL-ENC-005-RC16 Vendor Default Cipher Suites Used Without Review

The organization trusts vendor defaults for cipher suite selection and has not evaluated whether these defaults meet security standards.



Injection & Exploits (INJ)

Dynamic SQL in Stored Procedures

SEC-SQL-INJ-001-RC01 String concatenation for query building

Developers directly concatenate user input or variables into SQL strings using language string operators without parameterization, treating the resulting string as executable code.

SEC-SQL-INJ-001-RC02 Inadequate input validation and sanitization

Reliance on flawed or incomplete input validation, character escaping, or regex patterns that don't cover all injection vectors or can be bypassed by sophisticated attacks.

SEC-SQL-INJ-001-RC03 Lack of awareness and training

Nearly one-third of developers lack familiarity with secure coding practices; developers may not understand how SQL injection works or the risks of dynamic SQL without proper safeguards.

SEC-SQL-INJ-001-RC04 Legacy code and technical debt

Older stored procedures were written before parameterized query awareness; refactoring is postponed due to cost, time constraints, and fears of breaking dependent applications.

SEC-SQL-INJ-001-RC05 Time pressure and deadline constraints

Development teams under strict deadlines resort to shortcuts using simple string concatenation rather than implementing proper parameterized queries, which require more upfront effort.

SEC-SQL-INJ-001-RC06 Insufficient security code reviews

Code review processes may not specifically check for SQL injection vulnerabilities or dynamic SQL patterns; reviewers may lack security expertise or prioritize functionality over security.

SEC-SQL-INJ-001-RC07 Mixing static and dynamic SQL constructs

Developers build WHERE clauses or query fragments dynamically, then pass them to prepared statements, creating a false sense of security when the structure itself can be altered by attackers.

SEC-SQL-INJ-001-RC08 Complex business requirements

Multi-tenant systems, dynamic filtering, variable column selection, or reporting queries that require runtime table/column names appear impossible to implement with static SQL, leading developers to use dynamic SQL.

SEC-SQL-INJ-001-RC09 Performance optimization assumptions

Misunderstanding that dynamic SQL might improve performance in certain scenarios; using EXEC with concatenated strings to avoid perceived query plan caching issues (actually inferior to parameterized queries).

SEC-SQL-INJ-001-RC10 ORM framework misuse

Developers bypass ORM safety features by using raw SQL methods with string interpolation instead of the framework's parameterized query builders (e.g., FromSqlRaw, raw SQL in Hibernate/JPA without parameter binding).

SEC-SQL-INJ-001-RC11 Absence of security testing tools

No static code analysis, SAST tools, or runtime detection mechanisms in place to identify dynamic SQL patterns before production deployment.

SEC-SQL-INJ-001-RC12 Third-party library dependencies

Application relies on vendor code, legacy libraries, or abstraction layers that generate SQL dynamically; developers have limited visibility into actual SQL being constructed at runtime.

SEC-SQL-INJ-001-RC13 Operator control issues

Database operators or administrators use OS command execution features in maintenance jobs because built-in administrative tasks do not fully meet operational requirements.

SEC-SQL-INJ-001-RC14 Implicit trust in framework defaults

Developers assume that using stored procedures inherently prevents SQL injection without realizing that dynamic SQL within stored procedures remains vulnerable.

SEC-SQL-INJ-001-RC15 Incomplete parameter binding

Only some parameters are parameterized while others are concatenated; identifiers (table/column names) are concatenated



because they cannot be parameterized, creating partial vulnerability. **Developer/Code Root Causes:** - **Rapid prototyping mentality** - Build functionality quickly first, refactor later; dynamic SQL is faster to write initially but riskier long-term. - **Knowledge gaps in secure patterns** - Developers trained on how to use stored procedures but not how to implement them securely with dynamic SQL. - **Misaligned incentives** - Success metrics focus on features delivered, not security vulnerabilities prevented; security is treated as an afterthought. - **Copy-paste programming** - Legacy code examples, online tutorials, and internal code repositories contain insecure dynamic SQL patterns that developers replicate. - **Underestimation of attack scenarios** - Belief that database security is "someone else's problem" or that firewalls/network controls are sufficient protection.

Extended Stored Procedures in Use

SEC-SQL-INJ-002-RC02 **Backward compatibility requirements**

Multi-customer or long-running systems cannot easily drop support for OS command execution or other extended procedures without breaking dependent applications.

SEC-SQL-INJ-002-RC05 **High barrier to refactoring**

Migrating from OS command execution to alternatives requires database code changes, application code changes, infrastructure changes, and regression testing across all environments.

SEC-SQL-INJ-002-RC06 **Operational automation dependencies**

Database maintenance tasks (backup management, file system checks, log rotation, folder permissions) appear to require direct OS command execution from within the database.

SEC-SQL-INJ-002-RC07 **Insufficient privilege controls**

Developers grant administrative roles too broadly or fail to restrict who can enable/execute OS commands; the feature becomes accessible to accounts that should not have it.

SEC-SQL-INJ-002-RC08 **Missing monitoring and enforcement**

No detection mechanisms, audit policies, or triggers prevent unauthorized use or re-enabling of OS command execution after it has been disabled.

SEC-SQL-INJ-002-RC09 **Assumption of network isolation**

False belief that database servers are sufficiently isolated from threats; assumption that only trusted internal users can execute SQL injection attacks against database.

SEC-SQL-INJ-002-RC10 **Security compliance misalignment**

Compliance policies may require disabling OS command execution, but the control itself is ineffective because administrators with sufficient privileges can re-enable it at will.

SEC-SQL-INJ-002-RC12 **Third-party application requirements**

Vendor software or legacy applications explicitly require OS command execution; replacing or upgrading these systems is not feasible in the short term.

SEC-SQL-INJ-002-RC13 **Knowledge loss and staff turnover**

Original architects and developers who implemented OS command execution functionality have left; current teams lack documentation on why it was enabled and what depends on it.

SEC-SQL-INJ-002-RC16 **Insufficient removal validation**

When OS command execution is removed, hidden dependencies may exist in scheduled jobs, maintenance plans, or third-party tools; removal causes unexpected failures in production.



Logging (LOG)

Query Logging Missing

SEC-VEC-LOG-001-RC03 **pgaudit extension not installed/enabled**

PostgreSQL pgvector instances missing pgaudit extension; cloudsql.enable_pgaudit not set to "on"; pgaudit.log parameter not configured for READ/WRITE/ALL

Sensitive Query Leak

SEC-VEC-LOG-002-RC12 **pgvector logs containing SQL statements**

PostgreSQL log_statement=all capturing full SQL with WHERE clauses containing sensitive data; pg_audit capturing sensitive search conditions; log_min_duration_statement revealing slow queries with parameters



Networking (NET)

Database Exposed to rootcause Internet

SEC-SQL-NET-001-RC01 Bind Address Misconfiguration

Database configured to listen on 0.0.0.0 or all IPv4/IPv6 addresses instead of localhost (127.0.0.1) or specific private IPs. Default configurations in some setup guides recommend this for development convenience without emphasizing production security implications.

SEC-SQL-NET-001-RC02 Overly Permissive Security Group/Firewall Rules

Network access controls configured to allow inbound traffic from 0.0.0.0/0 (any IP) on database ports. Often created as temporary measures during troubleshooting that become permanent.

SEC-SQL-NET-001-RC03 rootcause Cloud Instance Configuration

Cloud-managed databases (AWS RDS, Azure Database, Google Cloud SQL) explicitly configured with "rootcause Access" or "rootcausely Accessible" enabled when this should be restricted to private VPC/VNet access.

SEC-SQL-NET-001-RC04 Inadequate Network Segmentation

No isolated network zones or VLANs separating database servers from rootcause-facing infrastructure. Flat network architecture treats all systems as equally accessible.

SEC-SQL-NET-001-RC05 Lack of IP Allowlist/Whitelist Enforcement

No explicit controls restricting database access to known, specific IP addresses or CIDR blocks of legitimate clients.

SEC-SQL-NET-001-RC06 Disabled or Absent Firewall Rules

Host-level or network-level firewalls either disabled entirely or configured with permissive default-allow policies rather than default-deny.

SEC-SQL-NET-001-RC07 Missing Virtual Private Network (VPN) Requirements

No requirement for clients to connect through VPN before accessing database. Database directly exposed without intermediary access controls.

SEC-SQL-NET-001-RC08 Cloud Provider Shared Responsibility Model Misunderstanding

Developers assume the cloud provider has secured all database access by default, not realizing they must explicitly restrict rootcause accessibility.

SEC-SQL-NET-001-RC09 Incomplete Infrastructure as Code Security

Terraform, CloudFormation, or other IaC templates contain insecure defaults that weren't reviewed before deployment. Modules copied from examples without security hardening.

SEC-SQL-NET-001-RC10 No Network Access Control List (NACL) Restrictions

AWS/Azure network ACLs not configured to restrict database port traffic, only relying on security groups which have been misconfigured.

SEC-SQL-NET-001-RC11 Shadow IT / Unauthorized Deployments

Database instances spun up by developers outside official deployment processes, bypassing security review and governance.

SEC-SQL-NET-001-RC12 Temporary Access Left Permanently

Security groups or firewall rules opened temporarily for debugging, testing, or rapid development never reverted to restrictive state.

SEC-SQL-NET-001-RC13 DNS Misconfiguration

Database hostname resolved to rootcause IP address instead of private/internal DNS name, allowing uncontrolled internet access.

SEC-SQL-NET-001-RC14 No Default-Deny Network Policy

Organizations didn't implement zero-trust or default-deny network architectures, relying instead on security awareness to prevent unauthorized exposure.

SEC-SQL-NET-001-RC15 Absence of Host-Based Firewalls

Database server running without local firewall rules restricting inbound connections. Relies solely on network-layer controls.

SEC-SQL-NET-001-RC16 Infrastructure Drift from Code

Database manually configured with rootcause access outside of IaC processes. Actual deployed configuration differs from what security reviews approved.



Default Ports in Use

SEC-SQL-NET-002-RC01 Default Configuration Never Changed

Database installed with factory defaults and administrators never modified the listening port. Databases ship with well-known default ports that are easily enumerated by attackers.

SEC-SQL-NET-002-RC02 Lack of Security Hardening Process

No post-installation security hardening checklist. Default ports considered "good enough" if protected by other means (which often aren't).

SEC-SQL-NET-002-RC03 Convenience and Standardization

Teams use default ports to reduce configuration complexity and ensure consistency across environments. Non-default ports require additional documentation and coordination.

SEC-SQL-NET-002-RC04 Documentation Focuses on Default Values

Deployment guides, tutorials, and quick-start documentation use default ports. Administrators follow these without considering security implications.

SEC-SQL-NET-002-RC05 Port Scanning Facilitation

Well-known ports make automated port scanning trivial. Attackers immediately know where to find services without port discovery phase.

SEC-SQL-NET-002-RC06 Application Connection String Hardcoding

Applications configured with connection strings pointing to default ports. Changing ports requires code changes, config file updates, and testing across all clients.

SEC-SQL-NET-002-RC07 Monitoring and Observability Tied to Defaults

Monitoring tools, dashboards, and alerting configured for default ports. Moving ports requires updating instrumentation.

SEC-SQL-NET-002-RC08 No Port Randomization or Obscurity Strategy

Organizations don't implement port randomization or assign non-standard ports. Treat port numbers as non-security-critical configuration.

SEC-SQL-NET-002-RC09 Infrastructure as Code Using Defaults

Terraform and CloudFormation templates use default port values without parameterization for environment-specific alternatives.

SEC-SQL-NET-002-RC10 Assumption Port Numbers Alone Provide Security

Administrators mistakenly believe that firewalls will block external access regardless of port number used, underestimating the impact of default port predictability.

SEC-SQL-NET-002-RC11 No Centralized Port Management

Organizations lack enterprise-wide port allocation standards, allowing ad-hoc default port usage across departments.

SEC-SQL-NET-002-RC12 Version Disclosure via Banners

Default ports combined with service banners reveal specific database software and versions, enabling version-specific exploitation.

SEC-SQL-NET-002-RC13 Brute Force Attack Simplification

Default ports reduce attacker reconnaissance time, enabling rapid brute force authentication attempts against commonly accessible database services.

SEC-SQL-NET-002-RC14 Lack of Port Rotation Practices

No periodic updates or changes to ports even when databases are patched or moved. Static port assignments become predictable targets.

SEC-SQL-NET-002-RC15 Automated Scanning Vulnerability

Internet-wide automated scanners probe default ports first. A significant portion of database breaches start with scanners finding services on default ports.

Unrestricted Outbound Connections

SEC-SQL-NET-003-RC01 Default Allow Firewall Posture

Operating system and network firewalls ship with default-allow outbound policies. Outbound traffic is rarely restricted unless explicitly configured as default-deny.



SEC-SQL-NET-003-RC02 Egress Filtering Not Implemented

Organizations focus resources on inbound protection (perimeter security) while neglecting outbound traffic controls. Egress filtering is less common and more complex than ingress.

SEC-SQL-NET-003-RC03 Database Features Requiring Internet Access

Features like automatic updates, telemetry, cloud backups, external data wrapper connections, or plugin repositories require outbound internet connectivity. Enabled by default without evaluating necessity.

SEC-SQL-NET-003-RC04 External data access configuration

Database features for querying external data sources (foreign data wrappers, linked servers, database links) establish arbitrary outbound connections. Configuration does not restrict allowed destination servers.

SEC-SQL-NET-003-RC05 External Data Source Integration

Databases configured to access external APIs, REST services, or data sources via UDFs (User Defined Functions) or stored procedures. No network policy restricting which external hosts can be reached.

SEC-SQL-NET-003-RC06 Package Manager and Repository Access

Database software accessing package repositories, pip/npm registries, or cloud storage for libraries and dependencies. No restrictions on destination servers or protocols.

SEC-SQL-NET-003-RC07 No Network Policy Enforcement at OS Level

Linux/Unix servers running databases without iptables/firewalld rules. Windows servers without Windows Firewall outbound rules.

SEC-SQL-NET-003-RC08 Cloud Provider NAT Gateway Configuration

AWS/Azure/GCP configured to provide internet access to database instances without explicit routing restrictions. Default behavior allows all outbound traffic.

SEC-SQL-NET-003-RC09 Lack of Web Proxy or Firewall Mediation

No proxy or firewall appliance between database and internet. Direct routing enabled for all protocols and destinations.

SEC-SQL-NET-003-RC10 DNS Unrestricted Outbound Access

Database servers with unrestricted DNS resolution capability to external servers, enabling initial reconnaissance and command-and-control communication.

SEC-SQL-NET-003-RC11 No Segmented Database Network Zone

Database in the same network segment as application servers, which require internet access for updates/APIs. Segmentation controls too permissive.

SEC-SQL-NET-003-RC12 Legacy System Requirements

Older databases or dependent applications required direct internet access for licensing, callbacks, or legacy protocol support. Continued even after modernization.

SEC-SQL-NET-003-RC13 Configuration Management Tool Access

Databases with access to configuration management systems (Puppet, Ansible) that require internet connectivity. No restrictions on which external systems can communicate.

SEC-SQL-NET-003-RC14 Database Replication to Cloud

Hybrid environments requiring outbound connections for replication to cloud databases. Implemented without restricting destination endpoints.

SEC-SQL-NET-003-RC15 Monitoring and Logging Exfiltration

Databases with unrestricted access to send logs, metrics, and telemetry to external monitoring services. No whitelisting of logging destination IPs.

SEC-SQL-NET-003-RC16 No Default-Deny Egress Policy

Organizations didn't implement zero-trust outbound policies. Standard practice is to allow unless explicitly denied.

SEC-SQL-NET-003-RC17 Lack of Connection State Inspection

Firewalls not configured for stateful inspection of outbound connections. Allow rules too permissive on protocols and ports.



Patterns (PAT)

End-of-Life Database Version

SEC-SQL-PAT-001-RC01 Lack of proactive EOL tracking and monitoring

Organizations fail to establish processes that monitor vendor EOL announcements, resulting in systems unknowingly drifting past support end dates without awareness or planning.

SEC-SQL-PAT-001-RC02 Deferred major version upgrades due to technical complexity

Major version upgrades require significant testing, potential data migration, application recompilation, or schema changes that demand substantial engineering effort, leading teams to postpone indefinitely.

SEC-SQL-PAT-001-RC03 High perceived costs of planned upgrades

Organizations prioritize short-term budget constraints over long-term risk mitigation, viewing upgrade expenses as capital expenditures rather than security investments.

SEC-SQL-PAT-001-RC04 Extended downtime requirements for migration

Organizations operating 24/7 systems fear the substantial downtime needed for major version migrations, particularly when zero-downtime solutions are unavailable or prohibitively complex to implement.

SEC-SQL-PAT-001-RC05 Application dependency on deprecated database features

Older applications may rely on deprecated database functions, syntax, or behavior no longer present in newer versions, requiring significant application refactoring or rewriting before migration is feasible.

SEC-SQL-PAT-001-RC06 Unfamiliar upgrade procedures and architectural changes

Teams lack expertise or documented procedures for their specific database engine and environment, creating fear of failed migrations or unexpected compatibility issues.

SEC-SQL-PAT-001-RC07 Inadequate testing environments that mimic production

Organizations cannot afford or maintain staging environments with representative data volumes and configurations, making safe upgrade testing impossible.

SEC-SQL-PAT-001-RC08 Resource constraints and competing priorities

Database teams spread thin across multiple systems lack bandwidth for planned upgrades, especially in organizations where new feature development takes precedence over maintenance activities.

SEC-SQL-PAT-001-RC09 Client and driver compatibility concerns

Applications using database clients or drivers compiled against older versions may require relinking, recompilation, or updates that demand coordination across multiple teams and application owners.

SEC-SQL-PAT-001-RC10 Loss of institutional knowledge

Original architects and engineers who designed the systems have left the organization, leaving no one with sufficient expertise to plan and execute complex upgrades safely.

SEC-SQL-PAT-001-RC11 Legacy application portfolio constraints

Some business-critical legacy applications were built on specific database versions and have not been maintained; attempting upgrades risks breaking these unsupported, unmaintained systems.

SEC-SQL-PAT-001-RC12 Incremental neglect over multiple release cycles

Organizations miss several consecutive EOL deadlines (e.g., "we'll upgrade from 11 to 12 eventually"), allowing the problem to compound until the cost becomes prohibitive.

SEC-SQL-PAT-001-RC13 Lack of automatic failure during deployment processes

No policies force systems to prevent deployment or operation on unsupported versions, allowing EOL systems to remain silently in production.

SEC-SQL-PAT-001-RC14 Vendor licensing and support service uncertainty

Organizations uncertain about licensing implications, support pricing for extended support, or cloud service EOL timelines delay decision-making. ****Organizational Root Causes:**** - ****Absence of strategic technology refresh planning**** – Organizations lack formal EOL roadmaps or capital planning processes that account for database lifecycle management as a regular operational cost. - ****Siloed responsibilities across infrastructure, application development, and security teams**** – No single team owns database lifecycle management, so responsibility falls through cracks with no accountability for proactive upgrades. - ****Reactive security**



culture** – Organizations respond to security incidents rather than proactively addressing known risks, treating EOL databases as acceptable until a breach occurs. - **Inadequate governance and compliance frameworks** – Lack of formal change control processes or compliance audits that would flag EOL systems as non-compliant and trigger action. - **Technical debt accumulation** – Years of deferred maintenance across infrastructure, applications, and processes create organizational inertia that makes large system changes feel impossible. - **Poor communication between business and technology** – Business leaders don't understand the security and operational risks of EOL systems, so budget requests for upgrades are frequently rejected. - **Understaffing in database administration and DevOps** – Organizations operate with minimal DBA or infrastructure staff, leaving no capacity for proactive work beyond firefighting production issues.

Missing Critical Security Patches

SEC-SQL-PAT-002-RC01 Incomplete asset discovery and inventory

Organizations lack comprehensive visibility into which database servers exist, their exact versions, and their current patch levels, making it impossible to patch systems they don't know exist.

SEC-SQL-PAT-002-RC02 Extended CVE disclosure-to-patch timelines

Vulnerabilities often remain unpatched for 30-90+ days between rootcause disclosure and organizational patch deployment, during which active exploitation occurs.

SEC-SQL-PAT-002-RC03 Inadequate testing environments and procedures

Organizations lack reliable staging environments that mirror production configurations, forcing teams to choose between untested deployment and delayed patching.

SEC-SQL-PAT-002-RC04 Fear of patch-induced breaking changes

Previous experiences with database patches introducing compatibility issues or breaking application functionality create organizational hesitation to deploy patches proactively.

SEC-SQL-PAT-002-RC05 Long approval workflows and change control processes

Multi-step approval hierarchies, compliance sign-offs, and change advisory board meetings delay patch deployment beyond the window of urgency.

SEC-SQL-PAT-002-RC06 Resource constraints and manual patching processes

Database teams cannot patch systems fast enough without automation, and lack of automation means slow, error-prone deployment.

SEC-SQL-PAT-002-RC07 Patch validation failures in staging

Some patches fail validation in test environments, requiring investigation, vendor escalation, or workarounds that consume time and delay production deployment.

SEC-SQL-PAT-002-RC08 Vendor patch delays for specific platforms or configurations

Some database vendors release patches slower for specific operating systems, hardware, or cloud environments, creating unavoidable delays.

SEC-SQL-PAT-002-RC09 Incompatible dependency chains

Applying a database patch may require updating dependent libraries, frameworks, or client drivers that themselves have unpatched vulnerabilities, creating a dependency resolution nightmare.

SEC-SQL-PAT-002-RC10 Active database usage preventing maintenance windows

Business demands for 24/7 availability eliminate maintenance windows needed for patch testing and deployment.

SEC-SQL-PAT-002-RC11 Lack of patch rollback and recovery procedures

Organizations uncertain how to safely rollback a failed patch deployment avoid patching altogether rather than risk system failure.

SEC-SQL-PAT-002-RC12 Critical patches for rarely-exploited vulnerabilities

Organizations suffer from alert fatigue when 90% of patches address theoretical vulnerabilities; prioritization failures lead to critical patches being missed among low-risk noise.

SEC-SQL-PAT-002-RC13 Incomplete patches that bypass exploits

Vendors release patches that address the reported vulnerability but leave alternative attack vectors open, requiring multiple patch cycles to fully resolve an issue.

SEC-SQL-PAT-002-RC14 Delayed CVE scoring and analysis

The National Vulnerability Database (NVD) and vendor security advisories sometimes lag weeks behind exploit availability, leaving



organizations unable to prioritize patching decisions.

SEC-SQL-PAT-002-RC15 Patch delivery and content distribution delays

Regional patch delivery networks or vendor infrastructure limitations delay patch availability in certain geographic locations or cloud regions. ****Organizational Root Causes:**** - ****Reactive patching culture driven by compliance audits**** – Organizations only prioritize patches when audits or regulatory reviews demand them, missing the actual threat window before patches are deployed. - ****Absence of formal SLAs for critical patch deployment**** – No defined targets (e.g., "critical patches within 48 hours") create ambiguity about urgency and accountability. - ****Organizational unwillingness to accept short-term operational disruption**** – Leadership resists scheduled maintenance windows that would provide time for testing and deployment, prioritizing uptime over security. - ****Insufficient staffing and training**** – Database teams lack expertise in patch management procedures, patch testing, and incident recovery, slowing deployment and increasing fear of failure. - ****Siloed security and operations teams**** – Security teams identify vulnerabilities but operations teams deploy patches; lack of coordination and shared accountability leads to delays and miscommunication. - ****Lack of automation investment**** – Organizations with manual, spreadsheet-based patch tracking and deployment processes cannot scale to modern vulnerability volumes and disclosure timelines. - ****Poor patch communication from vendor relationships**** – Organizations lack early access to security advisories, advance notification of critical patches, or vendor escalation channels for urgent issues. - ****Technical debt preventing safe patching**** – Legacy database configurations, undocumented customizations, or fragile application dependencies make patch testing and deployment extremely risky. - ****Competing priorities with limited budgets**** – Organizations allocate resources to new features and business initiatives rather than security maintenance, leaving patching perpetually underfunded. - ****Inadequate incident response and disaster recovery preparedness**** – Lack of confidence in rollback and recovery procedures increases organizational risk aversion to patching.



Priority & Scheduling (PRI)

Audit trail missing on personal-data tables

PPL Regulations §6 require logging access events to personal data, retained for at least 24 months. Detect databases without audit configured, or sensitive tables not covered by an audit specification.

SEC-SQL-PRI-005-RC01 No audit specification configured

Database engine has no audit specification, so no events are being recorded — Reg §6 requires logging access to personal data.

SEC-SQL-PRI-005-RC02 Audit configured but does not cover SELECT on PII tables

An audit exists but does not include SELECT actions on tables holding personal data. Read access is the most common exposure path.

SEC-SQL-PRI-005-RC03 Audit log retention < 24 months

Audit log files retained for less than 24 months, the minimum required by Reg §6 for High-level databases.

Backup hygiene & encryption

PPL Regulations §15 require regular, encrypted, recoverable backups of personal data. Detect missing recent backups, unencrypted backup files, backups stored on the same volume as live data, and short backup-history retention.

SEC-SQL-PRI-008-RC01 No recent backup of database holding personal data

No full / differential backup recorded in the last 7 days. Reg §15 requires regular backups of personal data.

SEC-SQL-PRI-008-RC02 Backups not encrypted

Most recent backups are stored without encryption. PPL Reg §15 requires backup encryption for High-level databases.

SEC-SQL-PRI-008-RC03 Backups stored on same volume as data files

Backup files reside on the same physical volume as the live database files — defeats the purpose of backup as disaster recovery.

Database security level not classified

Israel PPL Privacy Protection Regulations §2 require databases holding personal data to be classified as Basic, Medium, or High security level based on data sensitivity, number of data subjects, and authorised users. Detect databases lacking documented classification or whose configuration does not match the inferred level.

SEC-SQL-PRI-003-RC01 No documented security-level classification

Database has no extended-property / metadata indicating its PPL security level (Basic/Medium/High). Required by PPL Reg §2.

SEC-SQL-PRI-003-RC02 Database meets High-level criteria but configured as Basic/Medium

Database holds sensitive data of >100k subjects or >10 authorised users, which triggers High-level requirements. If classified lower, controls are likely insufficient.

SEC-SQL-PRI-003-RC03 External access enabled without High-level controls

Mixed-mode auth + remote access enabled but encryption / audit / strong auth not configured to High-level standard.

Excessive privileges on personal data

PPL Regulations §4–5 require least-privilege access to personal data. Detect public-role grants on PII tables, sysadmin/db_owner sprawl on application accounts, stale logins still having access, and direct grants bypassing role hierarchy.

SEC-SQL-PRI-006-RC01 Public role has SELECT on PII tables

Tables with sensitive columns are accessible to the public/PUBLIC role — meaning every authenticated principal can read them.

SEC-SQL-PRI-006-RC02 Application accounts hold sysadmin / db_owner

Accounts whose name suggests an application or service have sysadmin / db_owner / equivalent — violates least privilege.

SEC-SQL-PRI-006-RC03 Stale logins retain access to personal data

Logins with no recorded login activity for > 90 days still hold permissions on PII tables.



Israeli-specific sensitive data exposure

PPL §7 defines additional sensitive categories specific to Israeli context (Teudat Zehut / national ID, medical, religious, political opinion, sexual orientation, criminal record). Detect columns in these categories stored without encryption or with broad access.

SEC-SQL-PRI-011-RC01 Teudat Zehut (Israeli national ID) columns unencrypted

Columns matching teudat_zehut / tz / national_id / id_number patterns hold what is likely Israeli national IDs, stored without encryption.

SEC-SQL-PRI-011-RC02 Medical / health-data columns without classification

Columns matching medical / diagnosis / health / treatment / prescription patterns hold sensitive medical data per PPL §7.

SEC-SQL-PRI-011-RC03 Religion / political-opinion / sexual-orientation columns exposed

Columns matching religion / dat / political / orientation patterns hold sensitive opinion / belief / orientation data per PPL §7.

Personal data not encrypted at rest

PPL Regulations §9 require encryption of personal data at rest in High-level databases (and recommend it for Medium). Detect databases or sensitive columns stored without TDE / column-level encryption / encrypted backups.

SEC-SQL-PRI-004-RC01 TDE / database-level encryption not enabled

Database holds personal data but transparent data encryption (TDE) or equivalent database-level encryption is not enabled.

SEC-SQL-PRI-004-RC02 Sensitive columns stored without column-level encryption

Columns identified as PII/sensitive are stored as plain types without Always Encrypted / TDE column / pgcrypto / equivalent.

SEC-SQL-PRI-004-RC03 Connections not enforced over TLS

Server allows unencrypted connections; personal data in transit may traverse the network in plain text. Reg §9 requires encryption of data in transit when accessed remotely.

Personal data retained beyond purpose

PPL §14 and Regulations §14 limit retention of personal data to the period strictly necessary for the registered purpose. Detect very old records in PII tables, inactive customers with persistent records, soft-deleted rows never purged, and audit logs kept beyond legal minimum + safety margin.

SEC-SQL-PRI-009-RC01 Records older than retention threshold in PII tables

Tables containing personal data hold rows older than 7 years (default retention threshold). PPL §14 requires deletion when no longer needed for the registered purpose.

SEC-SQL-PRI-009-RC02 Inactive subjects with persistent personal data

Customer / user records with no recorded activity for >5 years still hold full personal data — should be archived or anonymised.

SEC-SQL-PRI-009-RC03 Soft-deleted rows never purged

Tables with is_deleted / deleted_at flags retain rows for >2 years after soft-deletion. Soft delete is not deletion under PPL §14.

PII Exposed in Clear Text

SEC-SQL-PRI-001-RC01 Misunderstanding of "Encryption at Rest" (TDE)

Teams often enable Transparent Data Encryption (TDE) or disk-level encryption (e.g., AWS EBS encryption) and believe the data is secure. This protects physical drives but *automatically decrypts* data for any valid user or application connection, leaving PII exposed to SQL injection or rogue DBAs.

SEC-SQL-PRI-001-RC02 Searchability \& Indexing Requirements

Applications often need to search, sort, or join on PII fields (e.g., "Find user by email"). Encrypting these columns (using randomized encryption) breaks indexing and search capability, so developers default to clear text to maintain performance.

SEC-SQL-PRI-001-RC03 Performance Overhead Fears

Developers avoid column-level encryption due to fears that decryption overhead will slow down high-throughput queries or increase CPU usage significantly.

SEC-SQL-PRI-001-RC04 Incorrect Data Types

Schema designers use standard `TEXT`, `VARCHAR`, or `CHAR` types for PII instead of `VARBINARY` or `BYTEA`. This forces the data to be stored as readable strings rather than the binary output of an encryption function.

**SEC-SQL-PRI-001-RC05 Legacy Schema Constraints**

The database was designed years ago without encryption. Altering the schema now to change column types and update all application queries to handle encryption/decryption is viewed as too high-risk or expensive (Technical Debt).

SEC-SQL-PRI-001-RC06 In-House "Masking" Logic Failure

Instead of true encryption, developers apply simple obfuscation (e.g., Base64 encoding or custom bit-shifting) which looks secure but is trivially reversible (clear text equivalent).

SEC-SQL-PRI-001-RC07 ETL & Data Warehousing Stripping

While the transactional database might encrypt sensitive columns, the ETL process that moves data to an analytics warehouse often decrypts it to simplify analyst queries, leaving the warehouse copy exposed.

SEC-SQL-PRI-001-RC08 Third-Party Integration Limitations

Off-the-shelf software or ORMs (Object-Relational Mappers) used by the organization may not support native database encryption functions, forcing clear text storage to ensure compatibility.

SEC-SQL-PRI-001-RC09 Key Management Complexity

The team lacks a secure Key Management Service (KMS). Because they don't know where to safely store the encryption keys, they hardcode them or avoid encryption entirely to prevent "losing the data" if keys are lost.

SEC-SQL-PRI-001-RC10 Lack of Native Masking Features (Old Versions)

Older database versions lacked native Dynamic Data Masking (DDM) features, leading to clear text storage as the default behavior unless custom views or application-level masking were built.

SEC-SQL-PRI-001-RC11 Unsecured Backups/Snapshots

Database backups or snapshots are taken without encryption. Even if the live DB is secure, a developer restoring a production snapshot to a lower environment exposes the clear text PII.

SEC-SQL-PRI-001-RC12 Sensitive columns discovered in schema (PII)

Database schema contains columns with names that strongly suggest PII or sensitive data (SSN, credit card, password, email, salary, etc.) that may not be encrypted, masked, or otherwise protected. These columns represent a compliance risk under GDPR, HIPAA, PCI-DSS, and similar regulations.

SEC-SQL-PRI-001-RC13 Sensitive columns actively queried (PII in use)

Columns identified as containing PII or sensitive data are actively being accessed by queries. This indicates live exposure risk — sensitive data is not just stored but actively read, potentially by applications, reports, or ad-hoc queries without proper masking or access controls.

SEC-SQL-PRI-001-RC14 Active transactions reading/writing sensitive columns (PII)

Currently-running queries reference tables that contain columns with sensitive names (password, ssn, credit card, email, etc.). Each row links a live transaction to a specific sensitive column it touches, including the executing login, host, program, and full SQL text. This is real-time PII exposure evidence — useful for compliance audits, DAM (Database Activity Monitoring), and forensic investigation of who accesses sensitive data.

SEC-SQL-PRI-001-RC15 Sensitive columns in cached plans / active transactions (PII)

Detects PII data exposure via two SQL Server DMV sources: (a) cached query plans in sys.dm_exec_query_stats — broader historical view of queries that have executed against sensitive tables, retained while the plan stays in cache; (b) currently-uncommitted transactions in sys.dm_tran_active_transactions — sessions holding locks or state on PII data. Each row links a query/transaction to the specific sensitive column it references, including execution count, average CPU/IO, transaction state, and full SQL text.

Security incident detection signals

PPL Regulations §11 require detection and notification of security incidents involving personal data. Surface signals that may indicate an incident: failed login spikes, privilege escalation events, unusual data export volume, and after-hours sensitive-data access.

SEC-SQL-PRI-010-RC01 Failed-login spike from a single source

>20 failed login attempts from the same host / login in the last 60 minutes — possible credential-stuffing or brute-force attack.

SEC-SQL-PRI-010-RC02 Privilege escalation event detected

Recent ALTER / GRANT events that elevated a non-admin login to sysadmin / superuser / db_owner.

**SEC-SQL-PRI-010-RC03 After-hours sensitive-data access**

Sessions querying tables with sensitive columns outside 06:00–22:00 local time — possible exfiltration or compromised account.

Sensitive Data in Logs**SEC-SQL-PRI-002-RC01 Global "Log Everything" Configuration**

Database query logging configured to capture all SQL statements, including those containing PII literals in INSERT, UPDATE, or WHERE clauses. Left enabled after debugging, it exposes sensitive data in plain text log files.

SEC-SQL-PRI-002-RC02 Unparameterized Queries (String Concatenation)

Developers build SQL queries using string concatenation (e.g., ``query` = "SELECT * FROM users WHERE ssn = " + ssn_var + ""`). This sends the literal PII value to the database log. Parameterized queries (using ``?`` or ``$1``) would only log the query structure, not the sensitive data.

SEC-SQL-PRI-002-RC03 Verbose Error Logging

Error logging configured to include the SQL statement that caused the failure. When a PII-laden query fails, the full query text with sensitive data is written to the error log.

SEC-SQL-PRI-002-RC04 Audit Trail Over-Configuration

Audit policies configured to capture full SQL text or bind variable values to meet compliance, unintentionally recording sensitive inputs in the audit trail.

SEC-SQL-PRI-002-RC05 Sensitive Functions in Logs

Even with some masking, specific functions like ``PASSWORD()``, ``AES_ENCRYPT('data', 'secret_key')``, or ``MD5()`` often bypass log redaction filters because the database logger sees them as standard function calls, logging the literal `'secret_key'`.

SEC-SQL-PRI-002-RC06 Crash Dumps and Core Files

When a database process crashes (e.g., Segfault), it writes a memory dump (core file) to disk. These dumps contain fragments of RAM which often hold clear text PII and passwords from recent queries.

SEC-SQL-PRI-002-RC07 Slow Query Logs with Low Thresholds

The "Slow Query Log" is enabled with a threshold of 0 seconds (or very low) to catch performance issues, effectively turning it into a general query log that captures data-heavy insert/update statements.

SEC-SQL-PRI-002-RC08 Client-Side "Debug" Modes

Drivers (JDBC/ODBC) or ORMs (Hibernate/Entity Framework) are configured to output generated SQL to standard output/files. These logs are often ingested by centralized logging tools (Splunk/ELK) alongside system logs.

SEC-SQL-PRI-002-RC09 Trace Files left Enabled

Database tracing or profiling tools enabled for debugging and left active after the session. These trace files capture query text and bind variable values containing sensitive data.

SEC-SQL-PRI-002-RC11 Insufficient Log Rotation/Permissions

Logs containing sensitive data are written to world-readable directories (``/tmp`` or ``/var/log``) and are not rotated/deleted frequently enough, allowing long-term accumulation of sensitive history.

Weak authentication / shared accounts

PPL Regulations §6 require identity verification of all users accessing personal data. Detect SQL logins without password policy, shared/generic logins (admin, app, svc), enabled SA login, and concurrent logins from multiple hosts.

SEC-SQL-PRI-007-RC01 SQL logins without password policy

SQL logins exist with `check_policy=OFF` / no password complexity, allowing weak passwords on accounts that may access personal data.

SEC-SQL-PRI-007-RC02 Shared / generic logins in use

Logins with generic names (admin, app, svc, dba, root) are active and have access to personal data, defeating identity attribution required by Reg §6.

SEC-SQL-PRI-007-RC03 SA / superuser login enabled and active

The instance's built-in superuser (sa / sys / postgres / root) is enabled and has been used recently — violates Reg §6 identity-verification requirements.



Vector Search (VS)

Default/Factory Passwords Active

SEC-SQL-VS-003-RC01 Failure to change default passwords during installation

Database administrators neglect to modify well-known default passwords immediately after database creation, leaving systems vulnerable to trivial attacks

SEC-SQL-VS-003-RC02 Databases upgraded from legacy versions preserving insecure defaults

Upgrades from older database versions preserve unlocked default accounts or weak security settings that were created before the vendor improved installation security.

SEC-SQL-VS-003-RC03 Applications dependent on default credentials

Third-party or legacy applications hardcode default passwords for accounts like DBSNMP, preventing administrators from changing them without breaking functionality

SEC-SQL-VS-003-RC04 Lack of automated password policy enforcement

The default password policy profile lacks complexity requirements, allowing weak passwords to persist indefinitely.

SEC-SQL-VS-003-RC05 Default accounts not properly locked

Accounts like CTXSYS, MDSYS, and DBSNMP remain unlocked after installation, contrary to security best practices

SEC-SQL-VS-003-RC06 Inadequate password verification functions

Failure to enable or customize password verification functions allows simple, guessable passwords that match usernames or follow predictable patterns.

SEC-SQL-VS-003-RC07 Fear of breaking database features by changing default passwords

Administrators hesitate to change default passwords for service accounts used by management tools, built-in components, or internal features due to concerns about service disruption.

SEC-SQL-VS-003-RC08 No regular password audits

Organizations fail to use default password audit tools or scanning utilities to identify accounts with default or weak passwords

SEC-SQL-VS-003-RC09 Insufficient account lifecycle management

Default accounts created during database feature or component installation are not tracked, reviewed, or properly configured with strong credentials.

SEC-SQL-VS-003-RC10 Weak password hashing in older versions

Legacy database versions use weak password hashing algorithms that can be cracked offline, making default or simple passwords especially dangerous.

SEC-SQL-VS-003-RC11 Default accounts granted DBA privileges

Many default accounts like DBSNMP and CTXSYS have powerful system privileges that can be exploited if compromised

SEC-SQL-VS-003-RC12 Lack of integration with enterprise password management

Database passwords managed separately from corporate password vaults or identity providers, leading to inconsistent security controls and credential sprawl.

SEC-SQL-VS-003-RC14 Inadequate documentation of required default accounts

DBAs lack clear guidance on which default accounts are necessary versus which can be safely locked, leading to conservative approaches that leave accounts open

SEC-SQL-VS-003-RC15 Default passwords in configuration files

Database configuration tools, installation scripts, and vendor documentation often reference default passwords, embedding them in operational processes and making them widely known.

Insecure Extensions Loaded

SEC-SQL-VS-002-RC01 Installation of untrusted procedural languages

Extensions like plpythonu, plperl, and pljavaU allow arbitrary code execution with superuser privileges, creating direct OS-level attack vectors



SEC-SQL-VS-002-RC02 Outdated extensions with known vulnerabilities

Failure to regularly update extensions exposes databases to rootcausely known CVEs, including privilege escalation flaws like CVE-2022-2625

SEC-SQL-VS-002-RC03 Lack of extension vetting before installation

Administrators install extensions without reviewing source code, checking CVE databases, or understanding security implications

SEC-SQL-VS-002-RC04 Superuser privileges required for extension installation

The CREATE EXTENSION command often requires superuser, forcing organizations to grant excessive privileges to users who only need specific extension functionality

SEC-SQL-VS-002-RC05 Extensions can CREATE OR REPLACE existing objects

Malicious actors can plant objects that extensions will adopt and grant superuser privileges to, enabling privilege escalation

SEC-SQL-VS-002-RC06 Insufficient supply chain verification

No cryptographic signing or integrity checking of extensions allows compromised or malicious extensions to be installed from untrusted sources

SEC-SQL-VS-002-RC07 Inadequate extension inventory management

Organizations lack comprehensive lists of installed extensions, versions, and their security postures, making vulnerability assessment impossible

SEC-SQL-VS-002-RC08 Extensions granted excessive privileges by default

Many extensions create functions with SECURITY DEFINER or similar high-privilege attributes that remain even when not needed

SEC-SQL-VS-002-RC09 Lack of runtime sandboxing

Untrusted languages execute without proper isolation, allowing direct filesystem access, network operations, and process execution

SEC-SQL-VS-002-RC10 No automated vulnerability scanning

Absence of tools to scan installed extensions against CVE databases or security advisories leaves known vulnerabilities undetected

SEC-SQL-VS-002-RC11 Developers installing extensions without security review

Database developers have authority to install extensions in development environments that later propagate to production without security assessment

SEC-SQL-VS-002-RC12 Extensions not included in patch management

Extension updates are often overlooked during database patching cycles, leaving security fixes unapplied

SEC-SQL-VS-002-RC13 Malicious extension objects can persist through backups

Trojan objects created by compromised extensions survive database dumps and restores, maintaining backdoor access

SEC-SQL-VS-002-RC14 Inadequate logging of extension-related activities

CREATE EXTENSION, ALTER EXTENSION, and extension function executions often lack proper audit trails

SEC-SQL-VS-002-RC15 Trusted extension mechanism bypasses security controls

The "trusted" flag allows non-superusers to install extensions that can still contain vulnerable or malicious code

Management UI/Database Privilege Overlap

SEC-SQL-VS-001-RC03 Overly permissive default role assignments

MC's predefined roles (Admin, Associate, IT, User) often grant broader privileges than necessary, violating least privilege principles by default

SEC-SQL-VS-001-RC05 Lack of regular privilege audits

Organizations fail to periodically review MC user assignments, mapped database accounts, and inherited privileges, allowing drift toward excessive access

SEC-SQL-VS-001-RC06 Insufficient training on dual privilege model

Database administrators may not understand how MC privileges interact with database-level grants, leading to misconfigurations

SEC-SQL-VS-001-RC07 Shared MC administrator accounts

Multiple users sharing the MC SUPER administrator account eliminates individual accountability and makes it impossible to enforce role-based access



SEC-SQL-VS-001-RC08 Stale user accounts retaining privileges

Former employees or transferred users often retain MC access and database mappings long after their legitimate need expires

SEC-SQL-VS-001-RC09 Inadequate privilege documentation

Lack of clear documentation on which MC privileges are required for specific job functions leads to over-provisioning

SEC-SQL-VS-001-RC10 No automated monitoring for privilege changes

Absence of alerting when MC users are granted new roles or mapped to different database accounts prevents timely detection of policy violations

SEC-SQL-VS-001-RC11 Inheritance of excessive privileges from mapped accounts

MC users mapped to database accounts with DBA or pseudosuperuser roles gain unrestricted access regardless of MC role assignments

SEC-SQL-VS-001-RC12 Use of default administrative credentials

Failure to change default MC SUPER administrator passwords creates a critical single point of failure

SEC-SQL-VS-001-RC13 Insufficient testing of privilege configurations

Organizations deploy MC privilege structures without validating that they enforce intended restrictions in practice

SEC-SQL-VS-001-RC14 Complexity in multi-cluster environments

MC's ability to manage multiple clusters with different security configurations creates inconsistent privilege enforcement across the infrastructure